Technische Universität Berlin

Distributed Systems
Chapter 5: Security, Kerberos and Blockchain

FG Verteilte Systeme und Betriebssysteme | Distributed Systems

Agenda

5.1 Communication Security

5.2 Authentication

5.3 Blockchain and Bitcoin

Security

Distributed Systems – A worst case security scenario

- Interfaces are open for cooperation reasons
- Unsecured networks: packets can be intercepted and manipulated, …
- Algorithms and program code for security-sensitive operations are known and available for attackers
- Access to comprehensive computing resources becomes more and more likely
- Spread of access information to several subsystems/locations

Security

Distributed Systems – A worst case security scenario

- Eavesdropping: copying of messages without permission
- Denial of Service
- Masquerading: usage of fake/foreign credentials
- Repetition: intercepted message is resent later, …

Computer vs. Communication Security

Computer Security:

- detection of compromised nodes and recovery from malicious states (e.g. a compromised server)

Communication Security:

- covers attacks against communication links and communicating entities

Terms and Definitions

Risk: potential unwanted and unexpected loss that may result from a given action

Vulnerability (physical or digital): flaw, gap, or weakness in IT environment or associated policies or processes that opens organization to potentially successful security breaches

Exploit: occurs when an attacker can take advantage of a vulnerability

Zero-Day Vulnerability: vulnerability that an organization is either unaware of or no patch exists yet → attacker may be able to more easily exploit this vulnerability until the organization is able to address it.

https://cloud.google.com/blog/topics/threat-intelligence/2024-zero-day-trends

Terms and Definitions

Security Breach: any incident resulting in unauthorized access to information or systems → attacker bypasses security mechanisms / controls.

Data Breach: type of security breach to steal confidential information.

Data Leak: sensitive information shared accidentally or intentionally with unauthorized parties without an attack → usually carried out by humans.

Cyber Threat: known, potential attack posing danger and risk to organization.

Cyber Attack: Threat carried out by an attacker

Terms and Definitions

Attacker: individual or organization that carries out cyber attacks (Cyber Criminals, Malicious Users, State-Sponsored Attackers, etc.)

Intruder: any attacker that has successfully gained unauthorized access

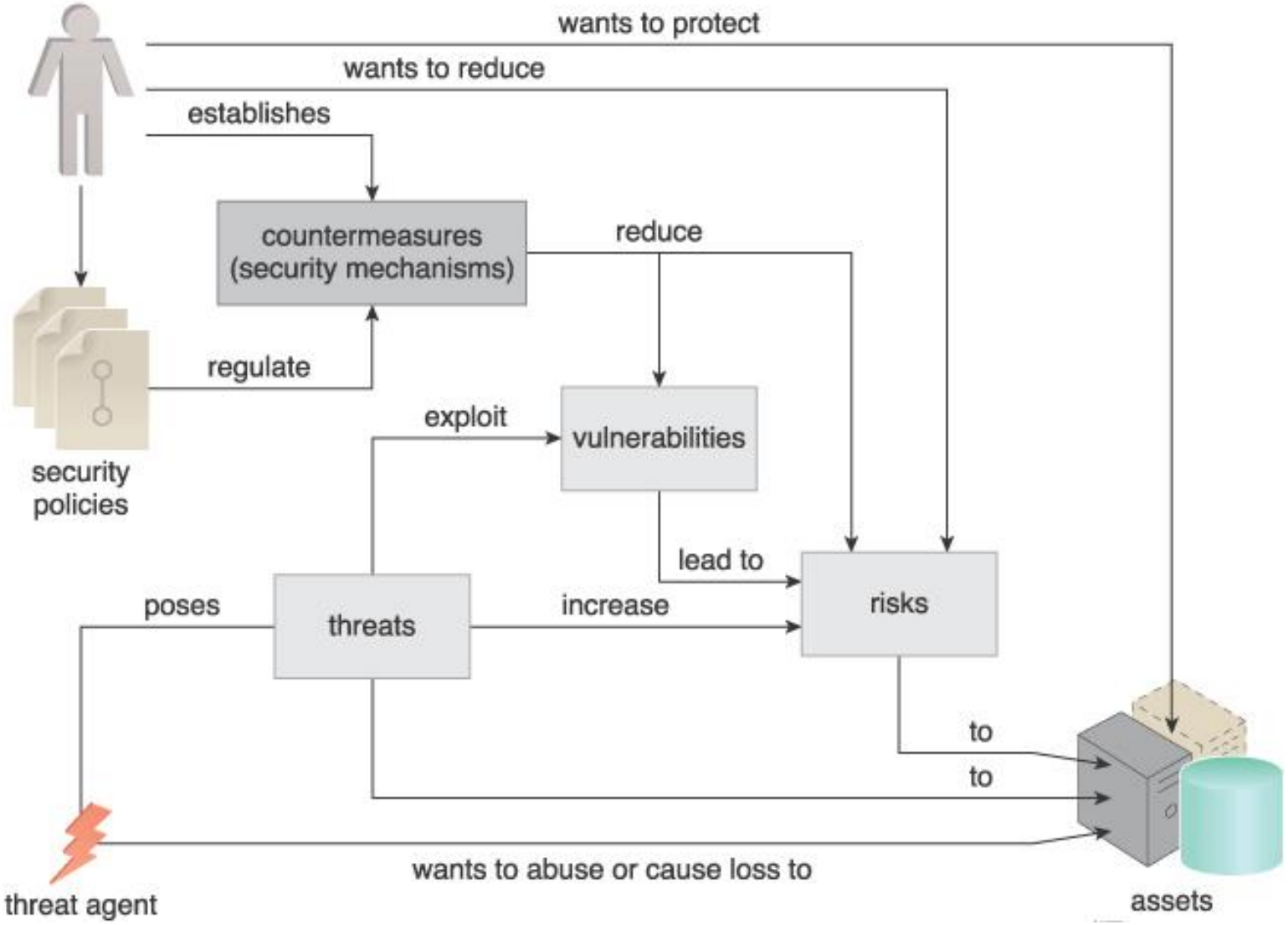
Attack vector: path that attacker takes to exploit vulnerabilities, e.g. email attachments, pop-up windows, chat rooms, instant messages. Human error or ignorance is commonly planned for when creating a given attack vector.

Attack surface: collection of attack vectors from which an attacker can access a system or extract information

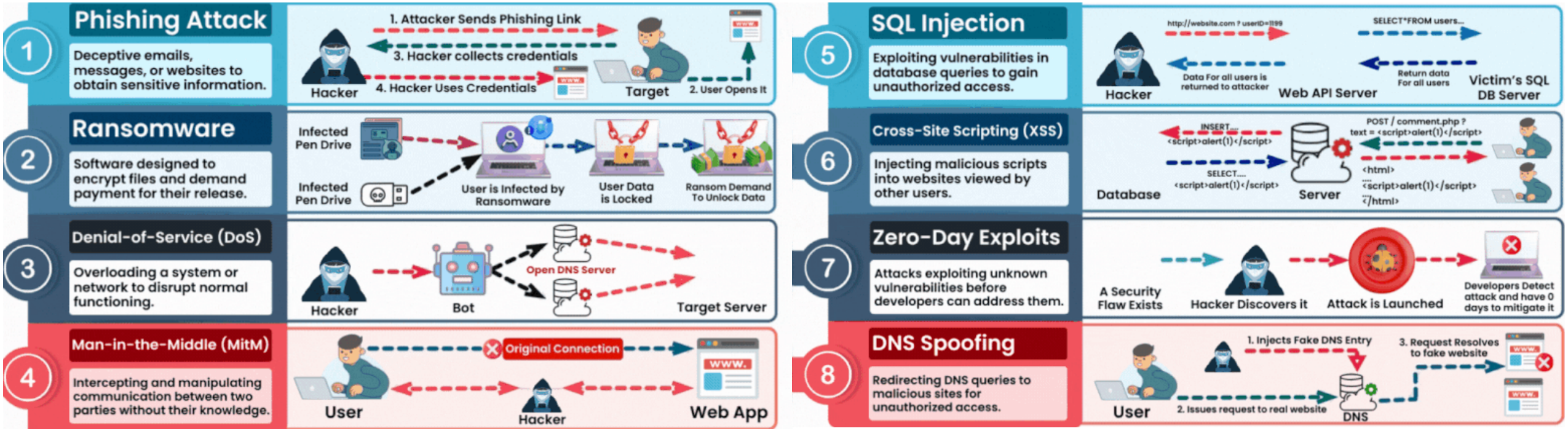
Threat agents

Entity: poses a threat because it is capable of carrying out an attack.

Security threats can originate internally or externally, from humans, AI or software



Top 8 Attack Types



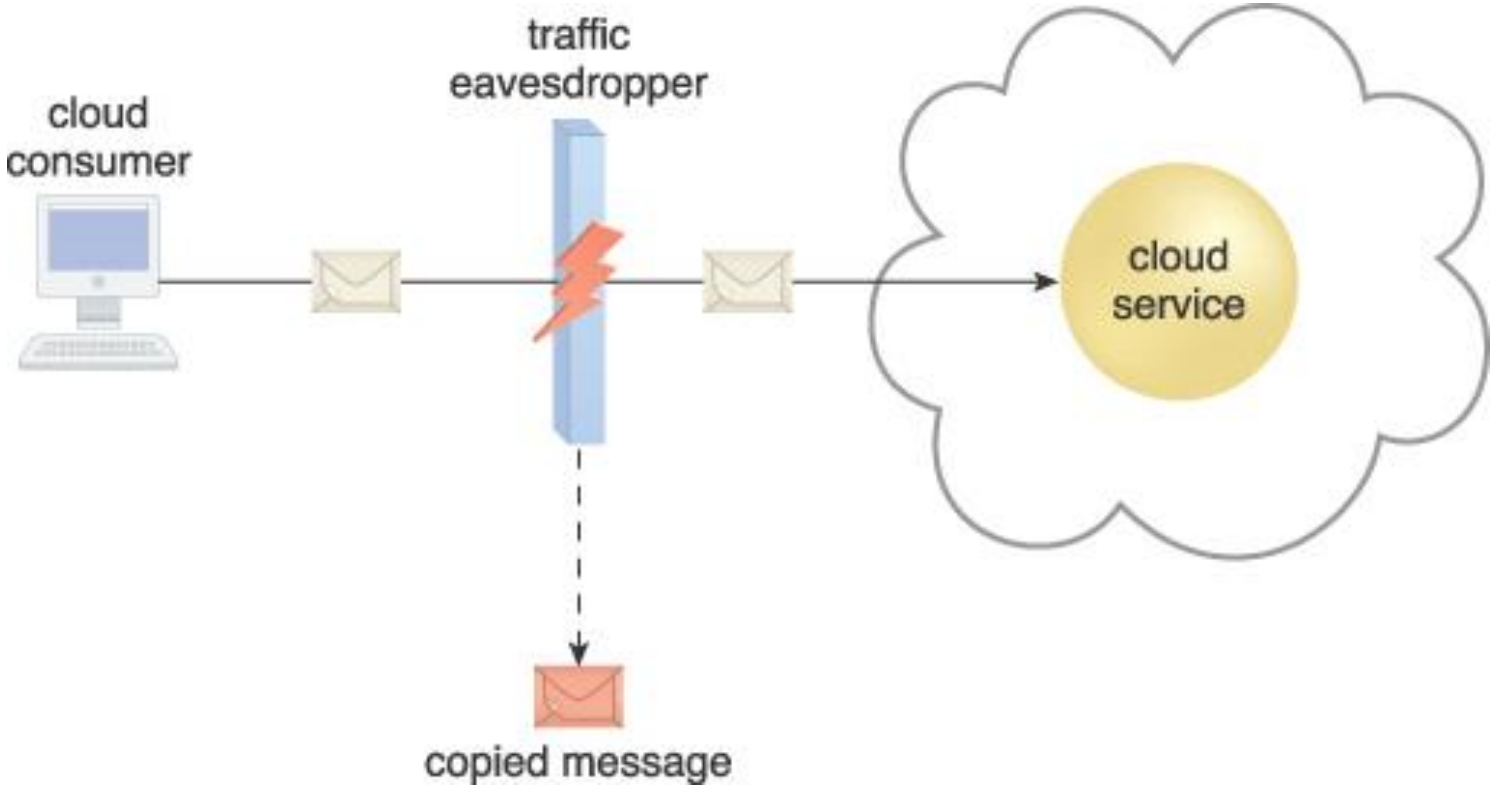
https://www.criacaodesites.com.br/os-8-principais-ataques-ciberneticos-e-como-se-prevenir/

Common Threats: Traffic Eavesdropping

Data transferred to or within DS passively intercepted by malicious service agent for illegitimate information-gathering purposes

Compromise the confidentiality of data and relationship between consumer and provider.

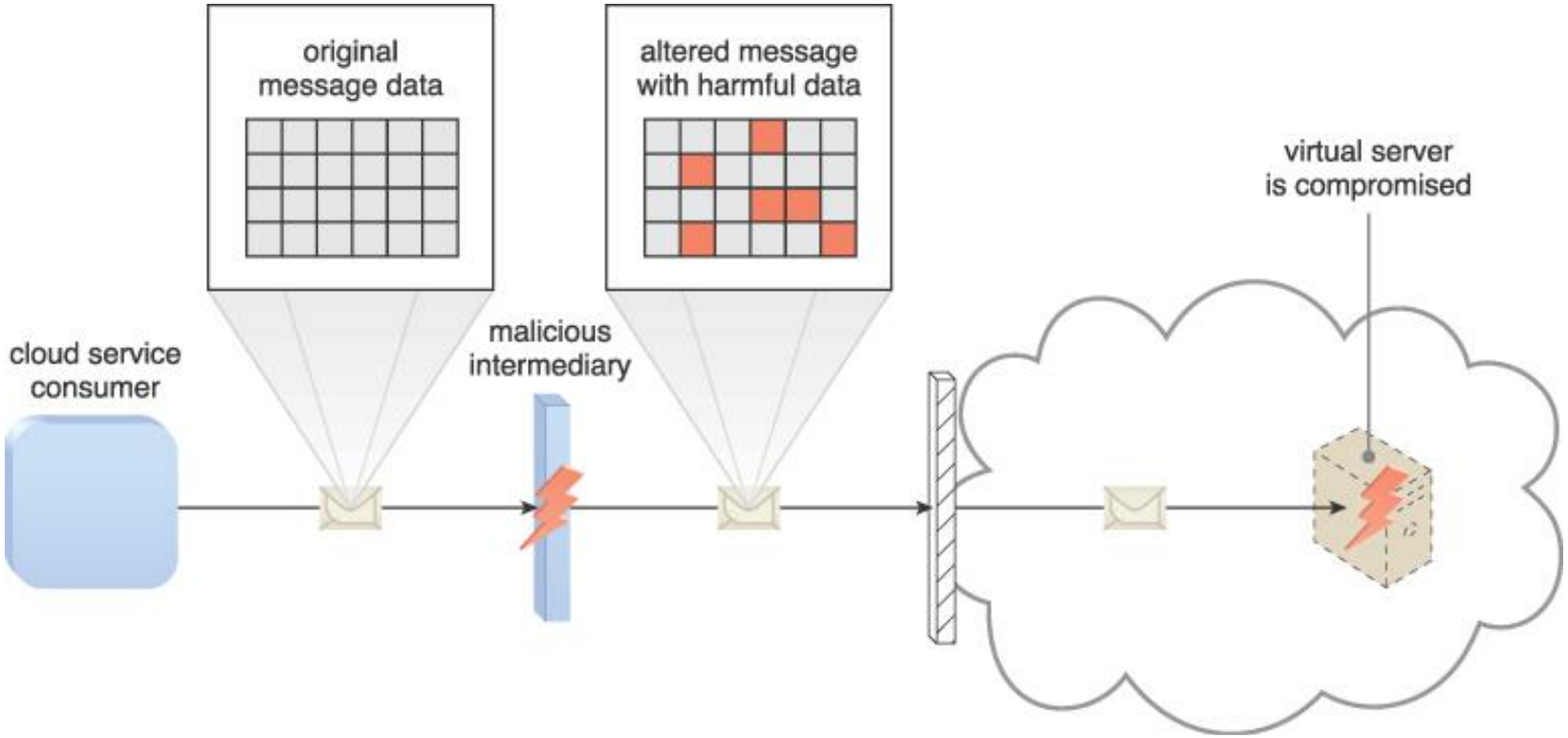
Passive attack nature → easily undetected for periods of time.



Common Threats: Malicious Intermediary

Messages intercepted and altered by malicious agent, potentially compromising confidentiality and/or integrity.

Inserting harmful data into the message before forwarding it to destination.



Common Threats: Denial of Service

Overload IT resources to the point where they cannot function properly, commonly launched in one of the following ways:

- Workload on services artificially increased with imitation messages or repeated communication requests.
- Network overloaded with traffic to reduce its responsiveness
- Multiple service requests sent, each of which is designed to consume excessive memory and processing resources.

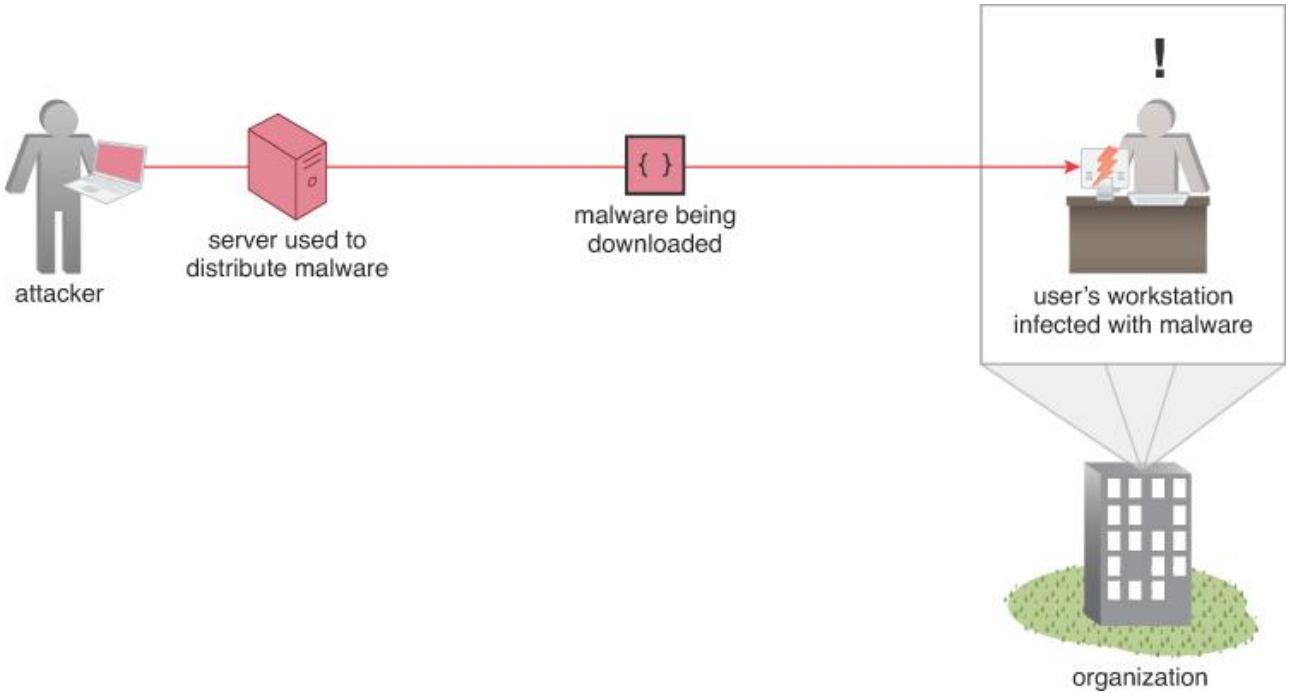
Successful DoS attacks produce server degradation and/or failure

Variant DDoS (distributed denial of service): multiple compromised systems used to flood targeted website or network with traffic to make it unavailable.

Common Threats: Malware

Malware (malicious software): software designed to cause harm to computer system or network, e.g.

- stealing protected data
- deleting confidential documents
- listening to private communication
- collecting information about confidential activity



Common Threats: Malware-based cyber attacks

Virus: code replicates and performs additional actions on the infected system.

Trojan: malicious software that appears to be a legitimate application or service.

Spyware: collects information about users or organizations without their knowledge.

Adware: displays unwanted advertisements or pop-ups

Ransomware: restricts/prevents data usage/access demanding payment to decrypt/release data

Bot: receives remote commands and reports information to source

Crypto Jacking: mine cryptocurrency without the user's consent.

Worm: self-replicating, self-propagating, self-contained program that uses network mechanisms to spread itself.

Common Threats: Social Engineering and Phishing

Social engineering: individuals tricked into revealing sensitive information or performing potentially damaging actions (e.g. granting access to unauthorized parties) → *Sometimes easier to exploit people than technology*.

Phishing: type of social engineering using electronic communication (e.g. fraudulent emails that appear to come from valid sources) to coerce users into

- releasing sensitive information
- triggering a security breach, or
- performing other damaging actions

Common Threats: Account attacks

Brute Force: attacker attempts broad range of possible username and password combinations → attacker gains unauthorized system access

Privilege escalation: attacker attempts to gain administrator permissions after compromising user account with limited access privileges by exploiting vulnerabilities that unintentionally allow a user account's access levels to be increased.

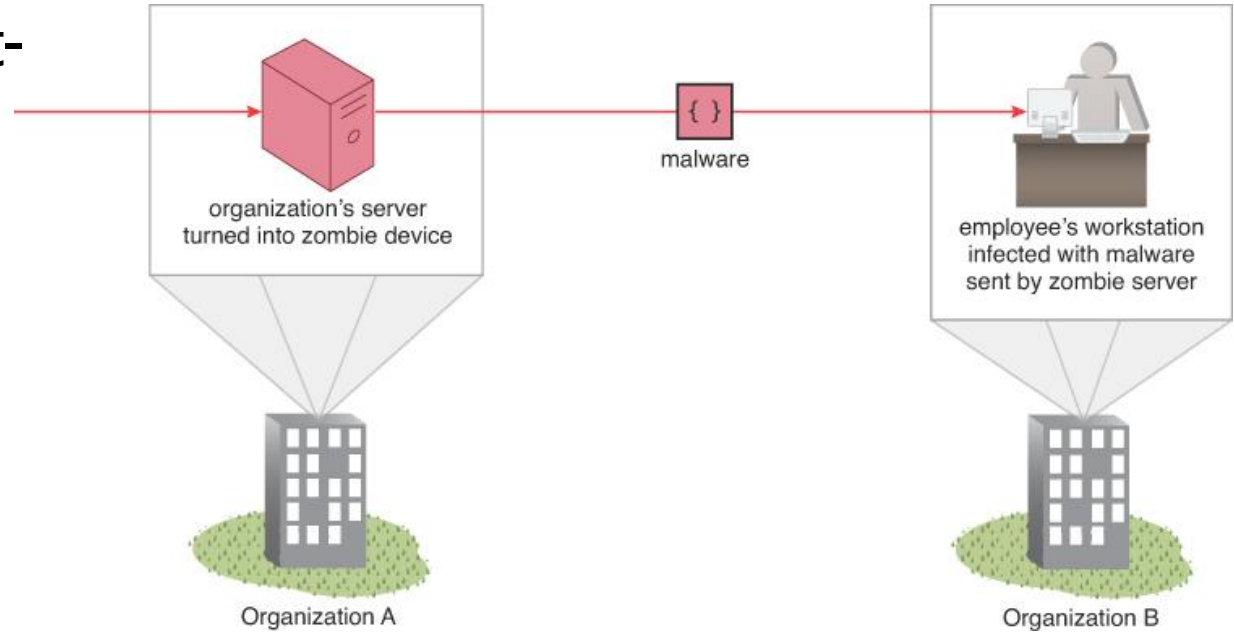
SQL injection: malicious code in SQL statements is inserted into an entry field of user interface, causing the web application server to execute the malicious code

Common Threats: Botnets

Attack via coordinated network of bots (bot-net) distributed across different hosts

- Initial infection creates "zombie" hosts = control over computer belonging to unsuspecting, legitimate organization → zombie host attacks another party
- Once installed connecting with bots on other infected hosts → network that attacker uses for malicious actions (e.g. DDoS, crypto jacking, recruiting new bots)

Botnets purchased/rented on dark web



Sample mechanisms to detect attacks

Hash-functions can be used to provide integrity checks

Message Authentication Codes (MAC) to check if the sender of a message is authentic

Digital signatures can be implemented based on Public Key Cryptography

Public-Key Infrastructuresa for reliable authentication

Public-Key Infrastructure (PKI)

How to ensure that the person I'm talking to is actually the one he claims to be?

How to validate that a digital signature corresponds to a specific entity?

- Certificates allow trustworthy relationship between a public key used for signatures and the owner
- Certificates combine information about the owner with the public key
- Contents and structure defined by X.509 standard:
  - Version and serial number
  - Signature algorithm ID (MD5, SHA-1, …)
  - Issuer Name
  - Validity Period
  - Subject Name and Public Key Info

Creating certificate

bash

1. openssl genrsa -out private.key 2048 (Generate a private key)
2. openssl req -new -key private.key -out request.csr (Create a Certificate Signing Request)
3. openssl x509 -req -in request.csr -signkey private.key -out certificate.crt -days 365 (Create a Self-Signed Certificate)
4. openssl pkcs12 -export -out certificate.p12 -inkey private.key -in certificate.crt (Bundle into a PKCS#12 (.p12) File)
5. openssl pkcs12 -info -in certificate.p12 (View Contents of the .p12 File)

PKCS#12 structure

Bag Attributes

localKeyID: BD 23 49 19 7E C9 41 4E 57 F4 25 48 38 2E ED B5 29 89 F1 D2 (Match private key)

subject=/C=DE/ST=BERLIN/L=BERLIN/O=TUB/OU=TU/CN=Odej Kao/emailAddress=odej.kao@test.de (Who the certificate is issued to)

issuer=/C=DE/ST=BERLIN/L=BERLIN/O=TUB/OU=TU/CN=Odej Kao/emailAddress=odej.kao@test.de (Issuer: Who issued the certificate)

C= Country; ST= State; L= Locality (City); O= Organization;

OU= Organizational Unit; CN= Common Name (typically name or domain)

Base64-encoded X.509 certificate

Encodes the public key, the subject, the issuer, validity dates, and digital signature.


```
-----BEGIN CERTIFICATE-----
MIIDeDCCAmACCQCZLSNcJ5dk4zANBgkqhkiG9w0BAQsFADB+MQswCQYDVQQGEwJERTEPMA0GA1UECAwGQkVSTE
lOMQ8wDQYDVQQHDAZCRVJMSU4xDDAKBgNVBAoMA1RVQjELMAkGA1UECwwCVFUxETAPBgNVBAMMCE9kZWogS2
FvMR8wHQYJKoZIhvcNAQkBFhBvZGVqLmthb0B0ZXN0LmRlMB4XDTI1MDcxMDA3MDI0NloXDTI2MDcxMDA3MDI0Nl
owfjELMAkGA1UEBhMCREUxDzANBgNVBAgMBkJFUkxJTjEPMA0GA1UEBwwGQkVSTElOMQwwCgYDVQQKDANUVUIx
CzAJBgNVBAsMAlRVMREwDwYDVQQDDAhPZGVqIEthbzEfMB0GCSqGSIb3DQEJARYQb2Rlai5rYW9AdGVzdC5kZTCCA
SIwDQYJKoZIhvcNAQEBBQADggEPADCCAQoCggEBANffWY7nCInCcL1MU8emYJ3f4xUcrl214ViutY8nWn+vjGJFeW6F
AZhoLwHZsquKuRBxzXMruWmy76d5qBPSEEoB2xkQTR87N68CkXq1myXcNlsAQ/cP987g7S/YTMcyu9u4V6m0FynUEf
mSaetB/mP7KDlQ9Br6ADwYGA5wcR9Ya8pgaYZkE3FS5jf4bAdELh9M90DG1j/K3+7l00Mker15qixtoDae2OwdsBGbo93
aR1HJxWbFB5YVipJjMJKFh01Z1Jsr9rZYWlCGkI7pk2aCcBZ/fss3QaJIbTBEOFoymRIqg11dS4wm0gGC7I1b1JZemExMS
b+Bwla/ngkTPGsCAwEAATANBgkqhkiG9w0BAQsFAAOCAQEAlCYPRpcbGG47Cqz7EorvoE0C8Dt7MkVXjiy6Y3Axc5Rq6
yMfQ3b19c1L8Pa3/rbYBqw+wwTLfOdLrsq+qSrwKmCxxqbNYADEvddCYThvfG5VUYubO6jhi6z55O7Y3ik79ZR8dkAfJ+
X5vQpgII8ceqfjhnHvJFeQGU+WPFfnX9I4jy7U0aWO9+maAyAdTvF/AUBNCc/JPxWDBt9E/9HpducKMejZXwEuE/VqjkO
KGJR2CFNj4aKxxHIyw3PdFbmLuVgLYJ0+ZcBOmR+8XYqv3NYqGhi1bNWue5hI3nwcyEa1Rq1KdtP+QBYq4+UeFcRM
MGx03utkMhek3ynyHTcNPQ==
-----END CERTIFICATE
```

Encrypted Private Key (PKCS#8 format, secured with symmetric encryption)

----Bag Attributes

localKeyID: BD 23 49 19 7E C9 41 4E 57 F4 25 48 38 2E ED B5 29 89 F1 D2

Key Attributes: <No Attributes>

-----BEGIN ENCRYPTED PRIVATE KEY-----
MIIFDjBABgkqhkiG9w0BBQ0wMzAbBgkqhkiG9w0BBQwwDgQI7UjLXQHnvgACAggAMBQGCCqGSIb3DQMHBAg1t7PvF5ImwASCBMgIa
zPpKb3FUfvTh1HQNxVLKd6ivj+48iDX1JRvp2VWY+RGvMZnQLzcUP05tcS3/dfREdljBQ9tTjjjU8cKPqonWCkT6nqEFWbXEtd2lkQwG6QTc
ewMn8V0U9KCAYhfwPMXbHHc3UXLVBur71Upl/HQRr+e/s2ARaXruD64M0VKSIZvbsTCrxOqLW5xCcHgq4lhXdii/lcPRGFIHQII2DqoID3V
NJeEVUB/izpp/yjePvKaOT0R0/q0RieKmqBMPhLjU+FGJweiMYC2qqE5EQM9gd/ijTGXUjHoWiohYpNwj8TGj/sFu/JB5sK36mv07QCZdWtf
FxNuN3gzx41ydZlJDLQ4gW+yeyUoyWgiVPDMp7vTmxvsDsScfNCTuks5pO2UIMTE1Pk3ndo1GMPh2DcBSjS4+ZB6LPtg517hDW0OCWV
nGuocsJW+sUUP4ApNIN8ooVNfcPw2DsYDxrxJOhA/1lQAn4Ofe4rFqSl4LPnCBHDqDWM+sOAIrcoN9MeIPKOuFiPsMgLA7JBn4zUP1+1e
Od9r1XEeAiiIj2QIuVzngSIbI5IOCvaj4kv+9aMCgI2rYI9b51uc3Bil5TIMkSANfW3b38TtRabZxAVvo95foc8/UKj/sO7XmrG+T4729ZoID7rC
0JU5of7139GtFbHoLJkFhC1nBPDxI9EvjaJ8YAvPDxgn9e5RjgmmbEsM5hpH3UopxYYkmP6Q4Ry3pw/MUTB8CoShVdMDlujPBg2Gae0uJ
wM8DAwaKUPHxPVuANhEcQr6KQ4M+ZEkmXoqhy/X1/qdrxtg2cKo9ce7WH0ZCZniFKSJt9TXuJCFa46qaI5YAuD/wcjjd1sw4hFLMCQZTq
4kzMFfESSJE+DR3B1Yzxvmf7mb8p7z05tlAUZMZb32+fqIrthWnN7TRdpNhgTn+DFNGLP/9Avv2djVcw5AC2MsAYmqgAGqEffqzOY0M2n
HEbYliMc9frn1PfoaEeGEKxD6UC0UYsisdJnFJ/5GJz/FlxZuQO/0CCRLGo4Q19AHNhHc6Jd+Xp+uo7QZKwrXfKiGDy69Vdh2sC7jKa0SHo0J
Mm2z7hhIV5ZsNQvh/6zO4WXw8LXQ8Bl4an83NM6mT/+1nGmY2r5vr6mU3XjbpEh+f+wSnxjbIyXAOKXeO49nZdax33cN8nWC7PtPYG
DAwwmFhBi91vPgRcApeWDCgIs9qAUptd2xPpv79J+fFMgPt8hABPjmZ8NRP+rCkCdmOKNFmrimd4Q+tbbL4Zg6jsZ+EHyPf9rslShZZXS
7xP4t86psACyV3M8VP6Ehy38XA0uoNNvPcAWcRsxG3suvZEgUaCIiqXs/FsMEoDeTHh2ZJ7ilFPIUEriZ9dTMDJ20TCzUxX9GSwE2j8nZDsS
FlZ656qxOjiWh/4kLAd1TrASb8GJsCqLK7QXbx/2OYZWrmS1x8hf385aX0hjbV/umroX2Ac4fiBcVluJTEvjzxOshF5gEzQeMrCnDJ+U2VqfBw
jTJwLKRbRy2c4+FGNHHHiCkJCSLjkh8Q1QSYdgSPj1O/2T7Y+P/x48kU1Fba70d41arhvFo95RA/1eKuwo9bExZG7lGiuvrGS9mEfbrASRlK
0ZmgOUu/qnJSCOnXgygXgGmNDNlscc=-----END ENCRYPTED PRIVATE KEY-----

Public-Key Infrastructure (PKI)

Certificates issued by a Trust Center or Certification Authority (CA)

User requesting certificate from CA

- generates a key-pair OR
- asks the CA to generate a pair

The user must identify herself, e.g. by presenting an ID

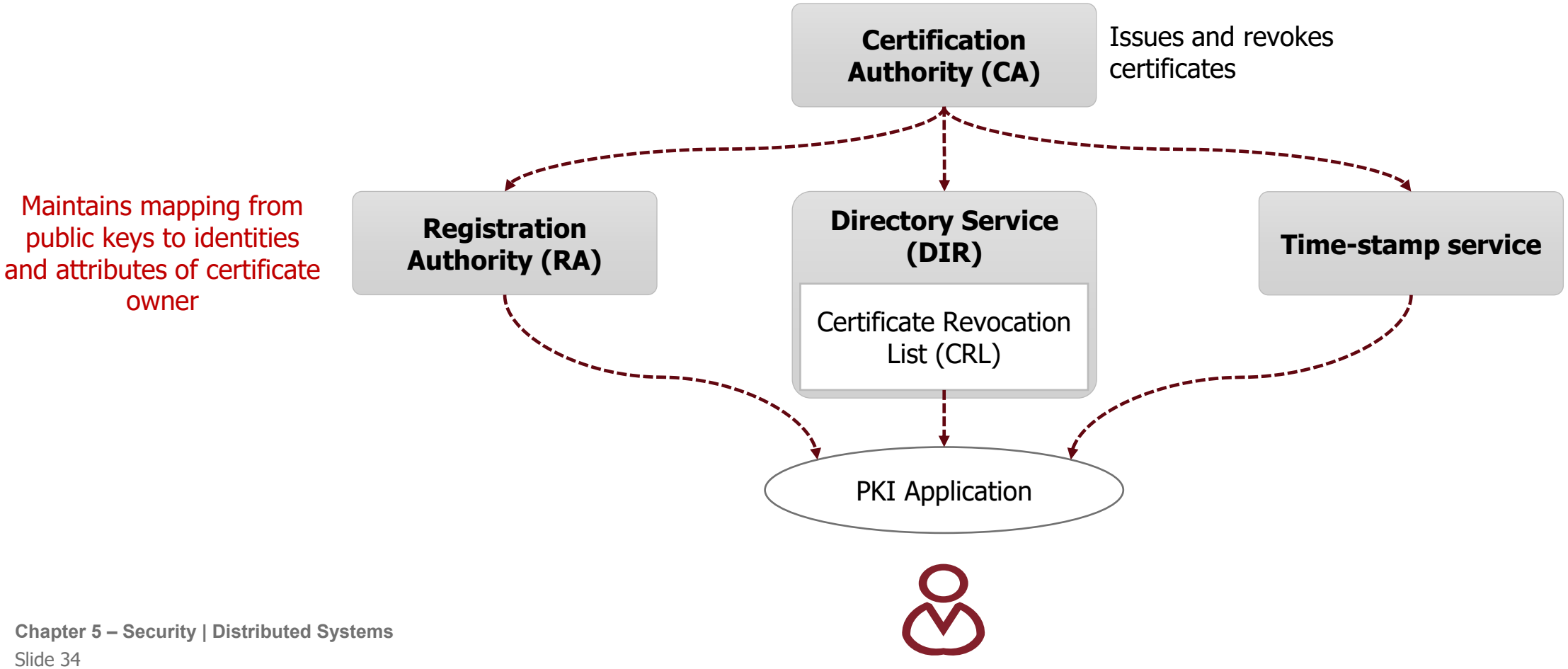
The CA creates the certificate and signs it using its own private key

- Each user trusting the CA now trusts the signed certificate
- Trusting means: user has imported the public key of the CA into its certificate store

Further responsibilities:

- Certificate revocation lists: provide access to lists with revoked certificates
- Recovery: in case of lost keys
- Time-stamp service: append a trustworthy time-stamp to data

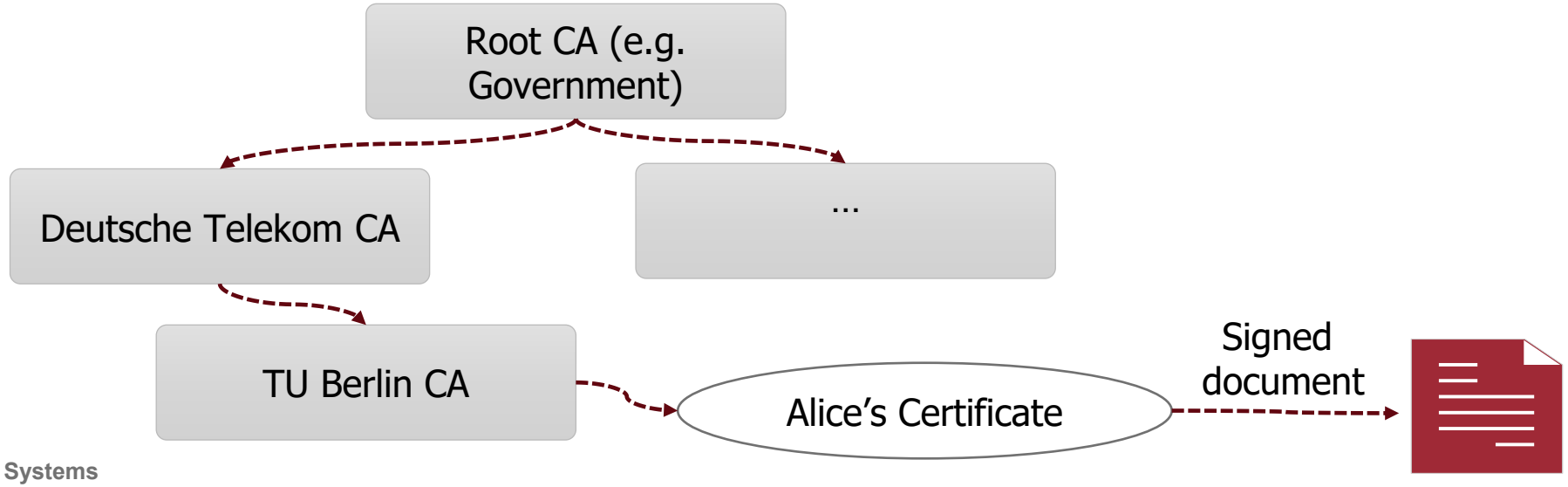
Public-Key Infrastructure (PKI)



Public-Key Infrastructure (PKI)

The validation of a certificate often follows a hierarchic approach

- Assume Alice sends Bob a document signed with her private key
- Given the certificate, Bob can validate the digital signature
- However, Bob needs to verify that the certificates actually belongs to Alice and is not faked

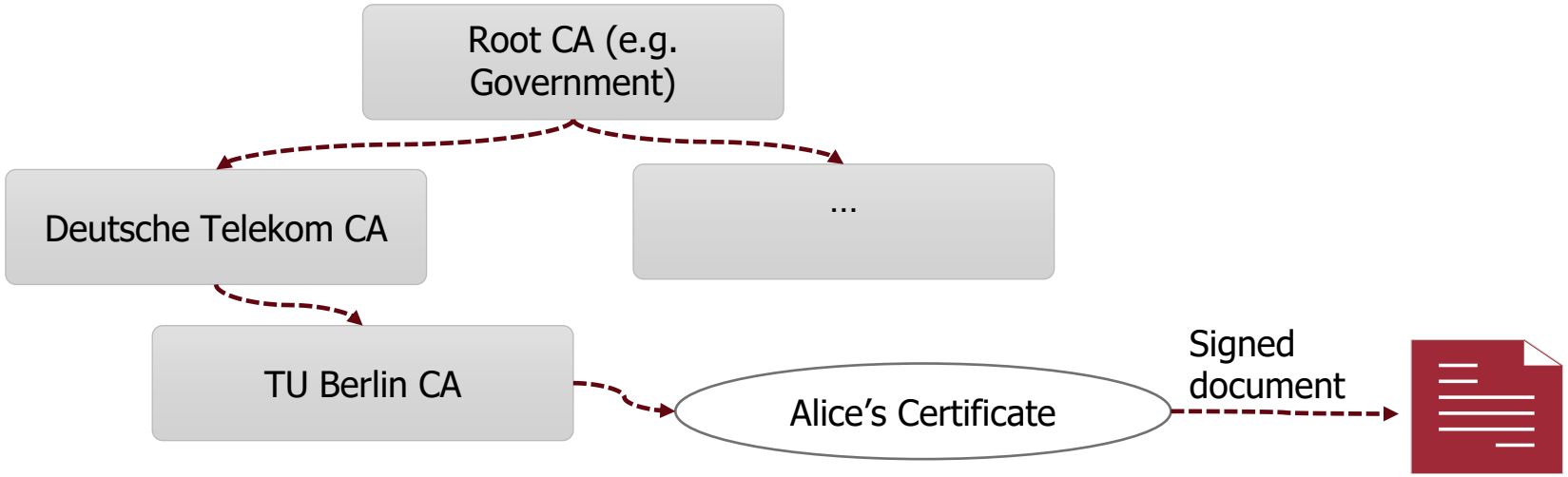


Public-Key Infrastructure (PKI)

The validation of a certificate often follows a hierarchic approach

- Bob computes hash of document and compares with hash decrypted with Alice public key contained in certificate
- Bob inspects certificate and finds that is was signed by TUB CA (compute and compare hash values using public key of TUB CA certificate)
- Process is repeated until Root CA reached → Bob trusts the root CA

Chain of Trust



Agenda

5.1 Communication Security

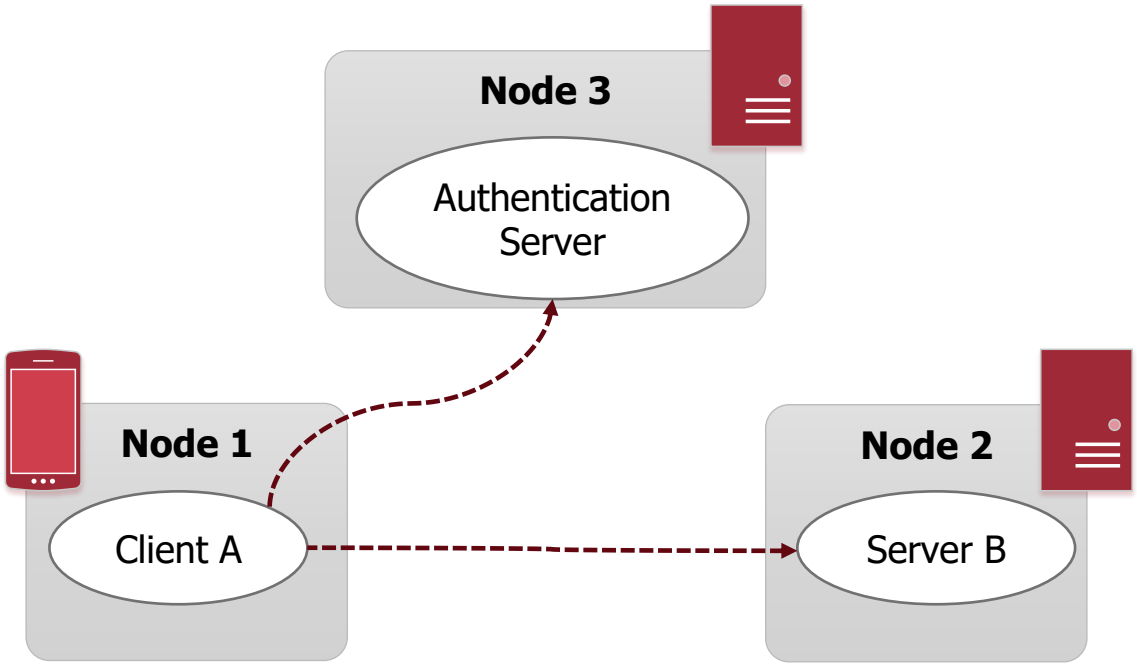
5.2 Authentication

5.3 Blockchain

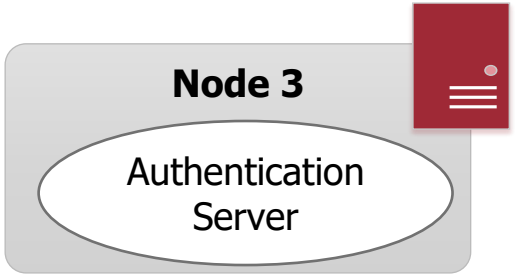
Needham – Schroeder (NS) Authentication Protocol

Questions: how to perform authentication between Clients and Servers *in Intranets* without revealing secrets?

- NS Protocol is based on an Authentication Server (AS) that is responsible for key distribution and authentication of principles



Needham – Schroeder Authentication Protocol



Tasks of Authentication Server (AS)

- Provision of a secure method for generating shared keys
- Distribution of secret keys to clients

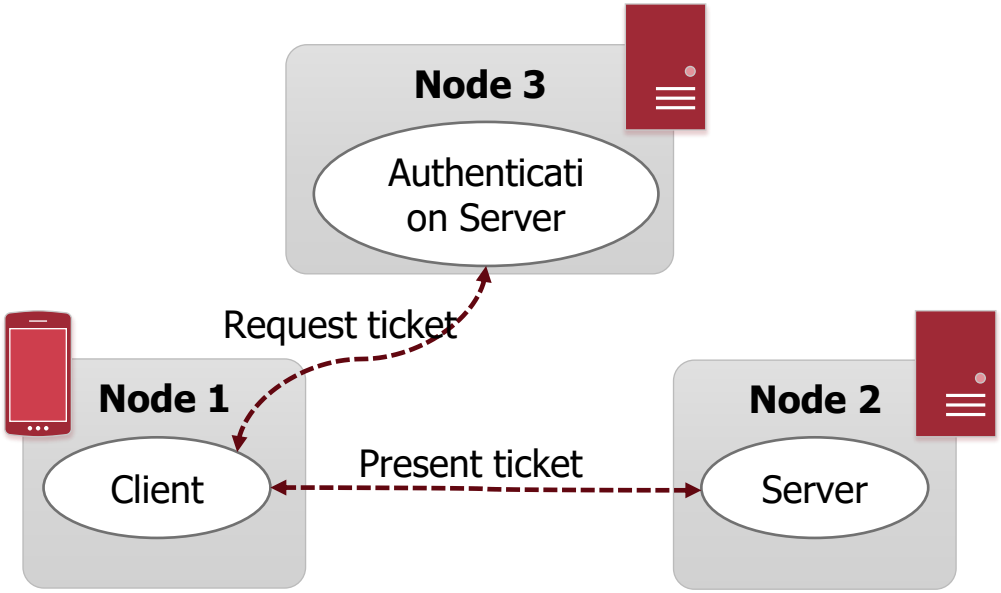
AS maintains a table with names and secret keys for each known principal

- Key serves as identification of the client with AS and secure communication between AS and client
- The key is never exposed to third parties and is transmitted across the network at most once – at generation

Needham – Schroeder Authentication Protocol

Upon request, the AS issues tickets that allow Clients to communicate with other entities

- The ticket gives evidence that the requesting client was authenticated by the AS
- Ticket: encrypted message with a secret key issued for communication between A and B



Needham – Schroeder Authentication Protocol

Header	Message	Notes
1. A → AS:	A, B, N_A	A requests AS to supply a key for communication with B.
2. AS→A:	$\{N_A, B, K_{AB}, \{K_{AB}, A\}_{K_B}\}_{K_A}$	AS returns a message encrypted in A's secret key, containing a newly generated key K_{AB} and a 'ticket' encrypted in B's secret key. The nonce N_A demonstrates that the message was sent in response to the preceding one. A believes that AS sent the message because only AS knows A's secret key.
3. A→B:	$\{K_{AB}, A\}_{K_B}$	A sends the 'ticket' to B.
4. B→A:	$\{N_B\}_{K_{AB}}$	B decrypts the ticket and uses the new key K_{AB} to encrypt another nonce N_B
5. A→B:	$\{N_B - 1\}_{K_{AB}}$	A demonstrates to B that it was the sender of the previous message by returning an agreed transformation of N_B.

Needham – Schroeder Authentication Protocol

Successful completion of protocol ensures:

- A and B are sure that they receive messages only from respective partner
- Messages are confidential (only A and B possess key necessary for decryption)

Weakness of the protocol:

- N_A, N_B nonces for proving the message freshness (nonce = random number used only once)
- B has no reason to believe that message 3 is fresh. If an intruder manages to obtain key K_{AB} and make a copy of the ticket $\{K_{AB}, A\}_{KB}$, he can later use A's identity
- Solution by adding a time stamp $\{K_{AB}, A, t\}_{KB}$ with a time limit
  - B decrypts this message and checks that t is recent
  - Adopted in Kerberos

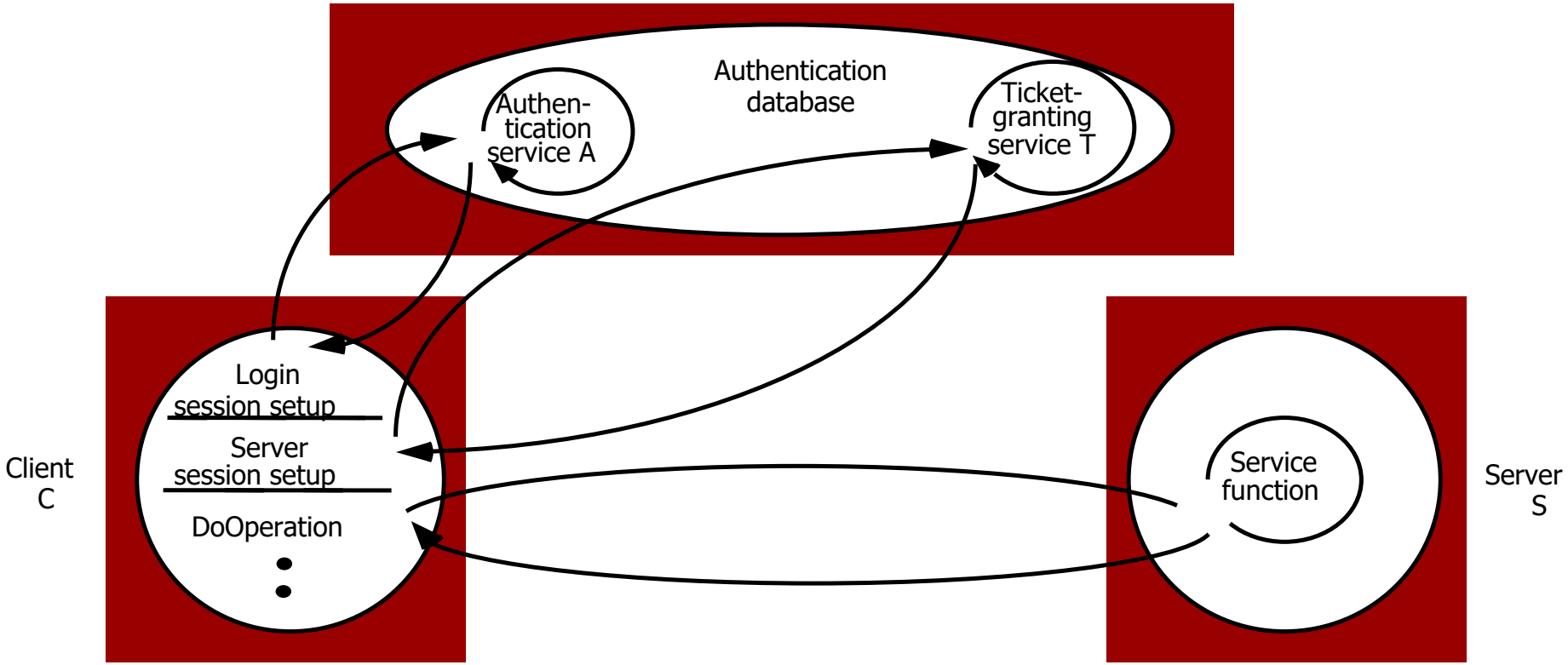
Kerberos

Based on the Needham-Schroeder Protocol, the Kerberos authentication protocol was developed by the MIT (Version 5 in RFC 4120)

Kerberos: Authentication of users/principals

- Servers demand a ticket from the client at every access (data access at NFS/AFS, sending of mails, registrations, print services, …)
- Passwords are known to users and Kerberos authentication service
- Services have secret keys which are only known to Kerberos and the servers providing the service

Kerberos


Authentication database
Authen-tication service A
Ticket-granting service T
Client C
Login session setup
Server session setup
DoOperation
Service function
Server S

Kerberos: Login procedure

Login: User sends the username to the Kerberos authentication service

If the user is known to the authentication service, it replies with

- A session key with a nonce, encrypted with the user's password
- A ticket for the ticket granting service (TGS)

Password is entered and stored locally

Login program uses password to decrypt the content of the reply

If successful → checks the nonce and saves the session key with the ticket for further communication with the TGS

Temporary copy of the password is deleted → passwords are never transmitted over the network

Kerberos

Accessing Servers – Servers require a valid ticket:

- Client program requests ticket for respective server at ticket-granting service TGS
- The received ticket is sent to the server with a new authentication (including timestamp) encrypted with the secret key of this service
- Server decrypts ticket, checks validity (life-time of ticket expired?) and extracts session key
- Session key is used for decrypting authentication $\Rightarrow$ Server checks whether the key is unused so far
- Login name/password of the user are not required for login

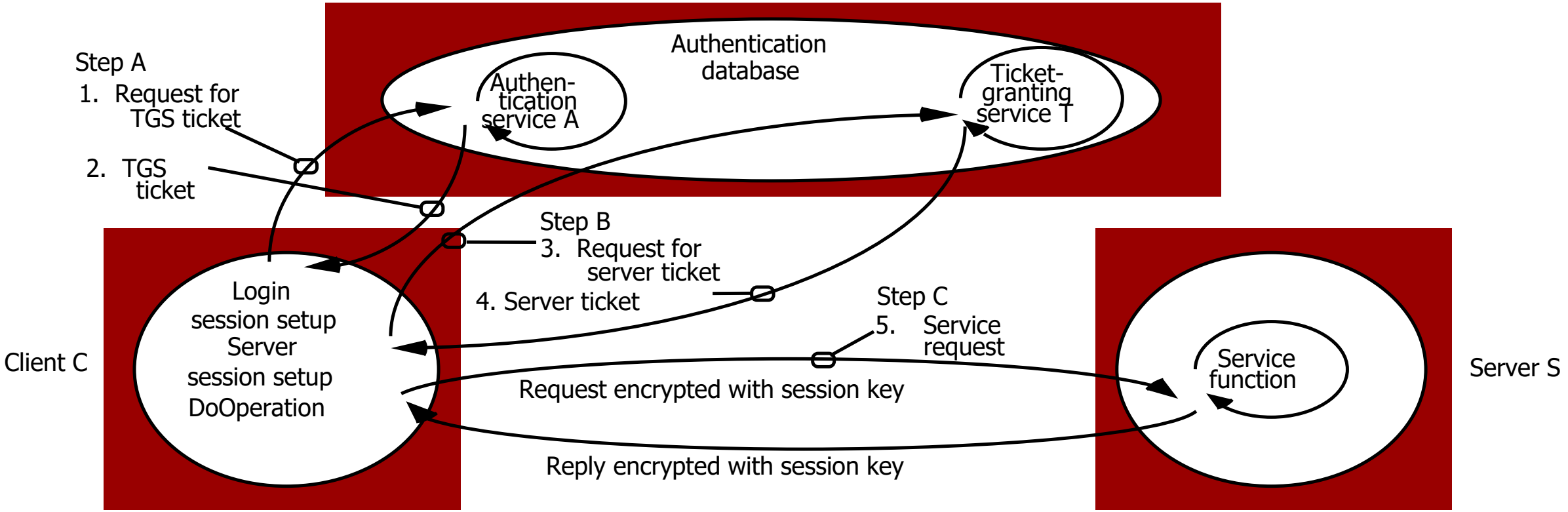
Kerberos

Works with three different kinds of security objects:

- Ticket: Token, issued by TGS service to clients
  - Presentation to server as proof that client is authenticated by Kerberos
  - Includes session key and expiry date
- Authentication: nonce, created by the client
  - Proves identity and that communication is up-to-date
  - Encrypted client name (with session key) and timestamp
- Session key: secret key generated by Kerberos, used for communication with a certain server

Clients must possess ticket and session key with lifetime of several hours for every service

Kerberos


Authentication database
Authen-tication service A
Ticket-granting service T
Step A
1. Request for TGS ticket
2. TGS ticket
Step B
3. Request for server ticket
4. Server ticket
Step C
5. Service request
Client C
Login session setup
Server session setup
DoOperation
Request encrypted with session key
Reply encrypted with session key
Service function
Server S

Kerberos

Kerberos Architecture is also called KDC (Key Distribution Center):

- Authentication service AS: authenticates user at registration ⇒ network secure password method
- Ticket Granting Service TGS: issues client tickets and session keys

Procedure following Needham-Schroeder-Protocol:

- Kerberos ticket for access of client C to server S in the form of $\{C, S, t1, t2, K_{CS}\}_{KS}$ with
  - t1 = ticket valid from this time
  - t2 = time when the ticket expires
  - K_{CS} = communication key between C and S
- Abbreviated notation $\{ticket(C,S)\}_{KS}$

Kerberos - Scenario

Request Kerberos session key and TGS ticket (once per login)

- Message 1 (C→KDC) = requests AS to issue a valid ticket for communication with TGS
  - Message is not encrypted, does not contain a password, just a nonce n for checking the validity of the response
  - Nonce n consists of date and time to avoid a) old messages (intercepted, might be manipulated) are newly transmitted and b) old tickets of logged-out users are reused
- Message 2 (Challenge, KDC→C) $\{K_{CT},n\}K_C$ with TGS session key, ticket
  - Encrypted with secret key of KDC based on password
  - Response $\{ticket(C, T)\}K_T$ with T = name of TGS, C = client name
  - Inclusion of nonce shows: message is from receiver of message 1, who must know K_C
  - This information is only useful if the receiver knows the secret key of client C (K_C)

Kerberos - Scenario

Request of a ticket for server S (once per client/server session)

- Message 3 (C→T) = request to TGS regarding ticket for communication with S, nonce n is re-issued and transmitted with the ticket
  - Encrypted authentication $\{C,t\}K_{CT} = \{auth(C)\}K_{CT}$
  - Transmission of the ticket $\{ticket(C,T)\}K_T$ for communication with TGS
- Message 4 (T→C) with ticket for server S
  - T checks the ticket → if valid, T generates a new random session key $\{K_{CS},n\}K_{CT}$ and returns it with a ticket $\{ticket(C,S)\}_{KS}$ for S
  - Encryption with secret key K_S of server S
- Now client C is ready to send requests to server S

Kerberos - Scenario

Client sends a request with a ticket to the server

- Message 5: (C→S) = request to S
  - C sends ticket with a newly generated authentication for C and a request to S
  - Request would be encrypted with K_{CS} if secrecy is required
  - Furthermore, a nonce is sent which is to be resent by server for securing authenticity
  - Ticket $\{ticket(C,S)\}_{KS}$ contains session key, which is used for decrypting the authentication
- Message 6: (S→C) with encrypted nonce $\{n\}_{Kcs}$

Agenda

5.1 Communication Security

5.2 Authentication

5.3 Blockchain

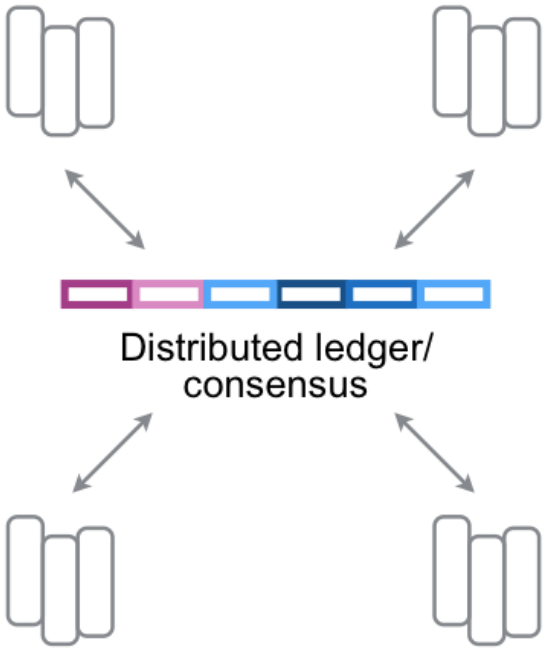
Definition Blockchain

Distributed database of records

Public ledger contains a certain and verifiable record of every transaction ever made

View on transactions/digital events are shared among participating parties

Global consensus



Decentralized, peer-to-peer

No trusted authority: bank, provider, certification authority, …

Weak identities

Everybody may use own software and implement such software him-/herself

No need for registration and real identity

Why know something on blockchain?

Digital currencies

Tracing: every product has a complete, trustworthy history, from origin, intermediate steps, status (Food chains, Replacement parts, Logistics, …) attached to it

Documents: life blockchain, passports, …

Trading: exchanging goods / currencies / … without central authority

Archiving in public government

Security in Internet of Things

Enterprise identity management and collaboration

Intellectual property in a digital age

Smart contracts: programs that automatically execute, when the terms of a contract, i.e. pre-configured conditions among entities, are met.

Bitcoin is one of the digital currencies

Introduced in 2008 by individual or group named Satoshi Nakamoto in the paper *Bitcoin: A Peer-To-Peer Electronic Cash System*

Transferring money

Traditional:

- Alice knows somehow Bobs IBAN → tells bank to transfer sum X to Bobs IBAN
- Bank checks Alice's identity (account number + PIN + mTAN), account balance, and plausibility of IBAN
- Alice bank issues transaction with Bobs bank and updates the account
- Bobs bank: Updates Bobs account

Bitcoin:

- Bob creates a transaction address and send it to Alice (Alice must not know the identity of Bob and Bob not the identity of Alice)
- Network verifies the account balance of Alice, updates Bobs account

Wallet and key pairs

Wallet stores the public key and associated private keys (or the seed to generate)

- Users must backup the file with the keys
- Send transaction to a specific Bitcoin address requires that the corresponding wallet knows the private key implementing it
- Loosing wallet → all coins are lost and can never be recovered

Where to send money? No IBAN, SWIFT → No master Bitcoin address

- New address for every transaction possible
- Everyone with the private key has access, identity not revealed
- Addresses added to key pool prior to being used for receiving coins

Number of bitcoin addresses: 1,46 x 10^48 (grains of sand on earth: 7,5 x 10^18)

Addressing

160-bit hash of the public portion of a public / private ECDSA key pair: Sign with private key, anyone with the public key can verify the signature

Standard P2PKH (Pay To Public Key Hash) for receiver's public key → 34 characters starting with 1, e.g. 1BvayiASVCmGmg4WUJmyRHoNevWWo5snqC

Approach

- Wallet inserts public key in hashing procedure starting with SHA 256 and then RIPEMD-160
- Insert Bytes 00 as prefix and 4 checksum bits at the end
- RIPEMD-160 hexadecimal hash converted in alphanumeric base58

Collisions possible (randomly created addresses) but unlikely: It is more likely that the Earth is destroyed in the next 5 seconds, than a collision in the next millennium

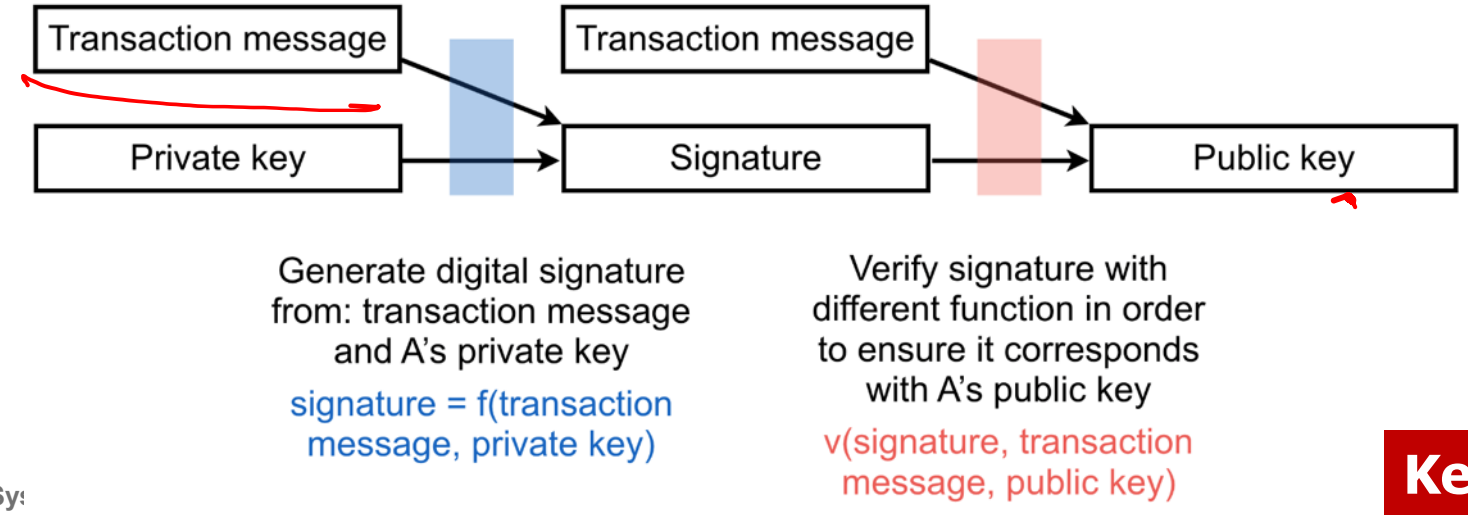
Starting transaction

1. Bob sends Alice his public key
2. Alice inserts the public key together with the amount to transaction
3. Alice signs the transaction with her private key
4. Everyone with Alice public key can verify that Alice acknowledged the transaction, as no one else has her private key
5. Only Bob can access the transaction sum, as he is the only one knowing the private key and allowed to insert the sum into a new transaction → Chain starts

How can transaction request be trusted

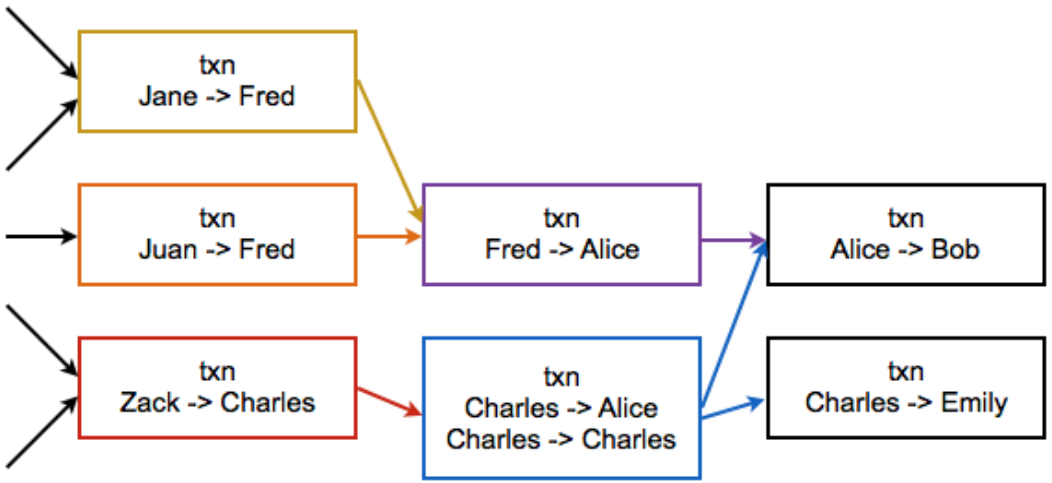
How to know that a transaction request is authentic, i.e. the rightful owner (Alice) sent the message?

Digital signature: unique per transaction, cannot be reused, verifies the integrity of the message



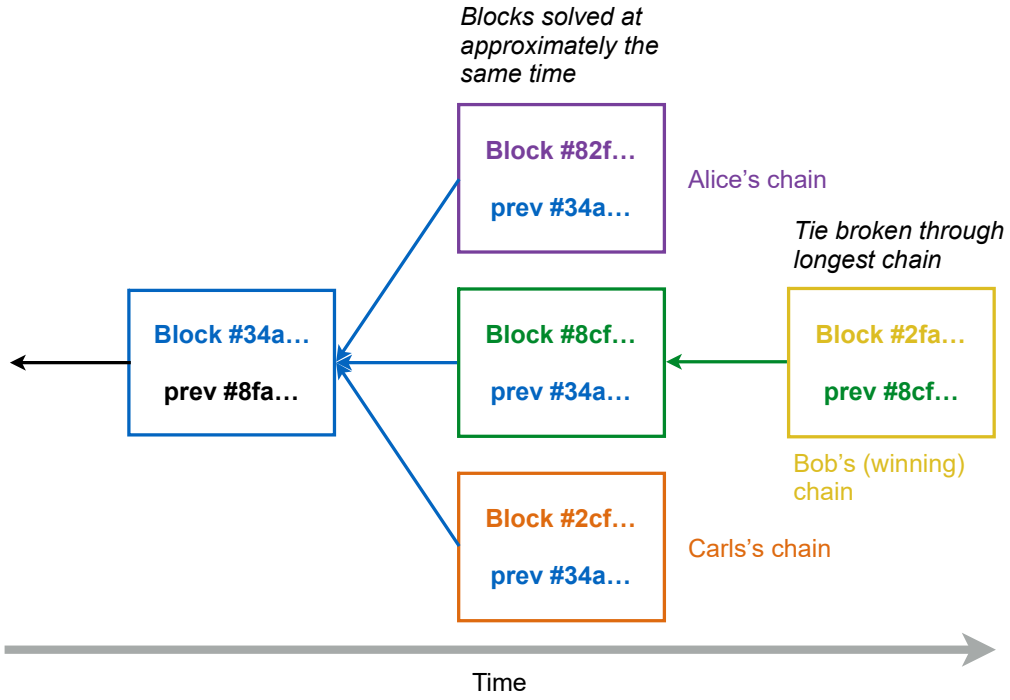
Kennen wir schon

Transaction vs. blockchains



Transaction graphs: transfer of ownership

Blockchains: ordering of transactions



Executing transactions (i.e. sending money)

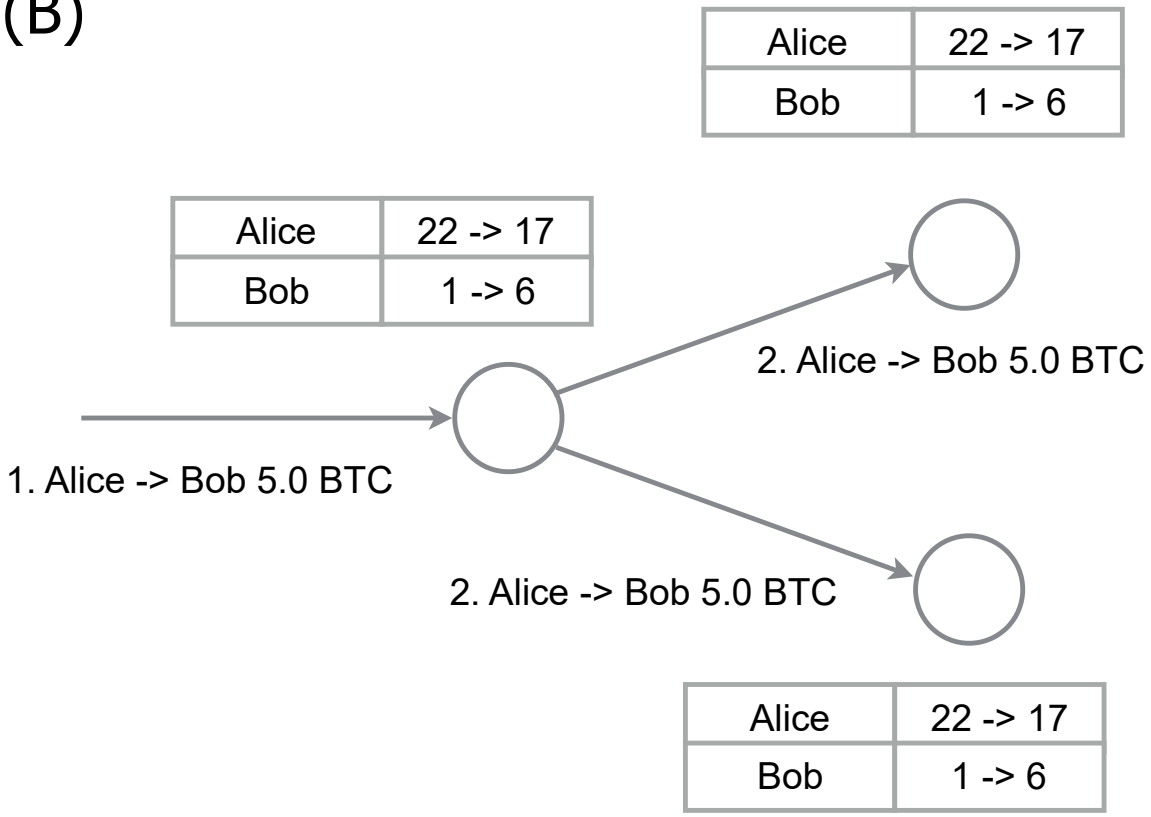
Transaction: send x from Alice (A) to Bob (B)

Broadcast to network x deducted from A's and added to B's account

Every node in network

- receives request
- applies to own ledger
- pass to other nodes

Implication: everyone knows about everyone's else's transaction



Transaction as double-entry bookkeeping

Ownerships of funds verified through links to previous transactions

Transaction as Double-Entry Bookkeeping

Inputs	Value	Outputs	Value
Input 1	0.10 BTC	Output 1	0.10 BTC
Input 2	0.20 BTC	Output 2	0.20 BTC
Input 3	0.10 BTC	Output 3	0.20 BTC
Input 4	0.15 BTC		
Total Inputs:	0.55 BTC	Total Outputs:	0.50 BTC

	Inputs	*0.55 BTC*
-	*Outputs*	*0.50 BTC*
	Difference	*0.05 BTC (implied transaction fee)*

Transaction 7957a35fe64f80d234d76d83a2a8f1a0d8149a41d81de548f0a65a8a999f6f18

INPUTS From

From (previous transactions Joe has received):
Joe 0.1005 BTC

OUTPUTS To

Output #0 Alice's Address 0.1000 BTC (spent)
Transaction Fees: 0.0005 BTC

Transaction 0627052b6f28912f2703066a912ea577f2ce4da4caa5a5fbd8a57286c345c2f2

INPUTS From

7957a35fe64f80d234d76d83a2a8f1a0d8149a41d81de548f0a65a8a999f6f18 : 0
Alice 0.1000 BTC

OUTPUTS To

Output #0 Bob's Address 0.0150 BTC (spent)
Output #1 Alice's Address (change) 0.0845 BTC (unspent)
Transaction Fees: 0.0005 BTC

Transaction 2bbac8bb3a57a2363407ac8c16a67015ed2e88a4388af58cf90299e0744d3de4

INPUTS From

0627052b6f28912f2703066a912ea577f2ce4da4caa5a5fbd8a57286c345c2f2 : 0
Bob 0.0150 BTC

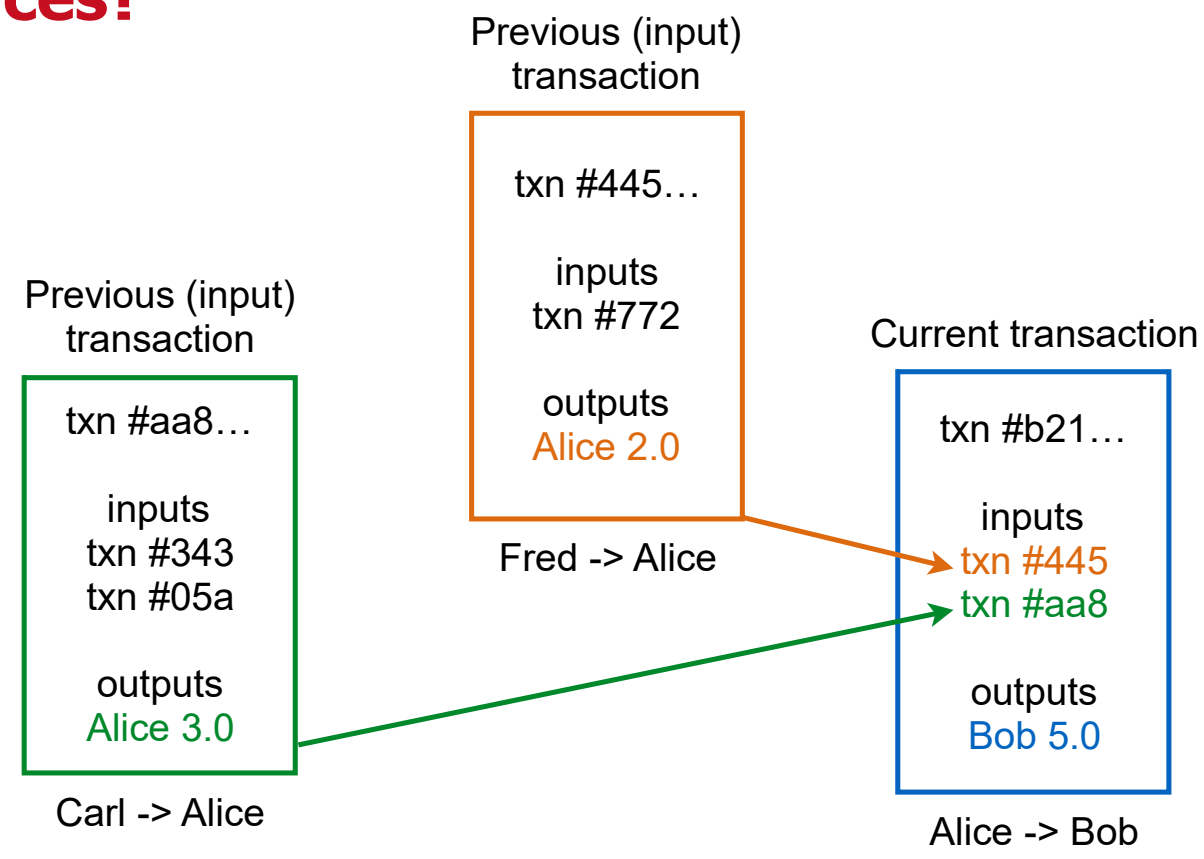
OUTPUTS To

Output #0 Gopesh's Address 0.0100 BTC (unspent)
Output #1 Bob''s Address (change) 0.0845 BTC (unspent)
Transaction Fees: 0.0005 BTC

How to keep track of account balances?

A want to send x BTC

A must reference transactions,
where A received y BTC
from someone else: previous inputs



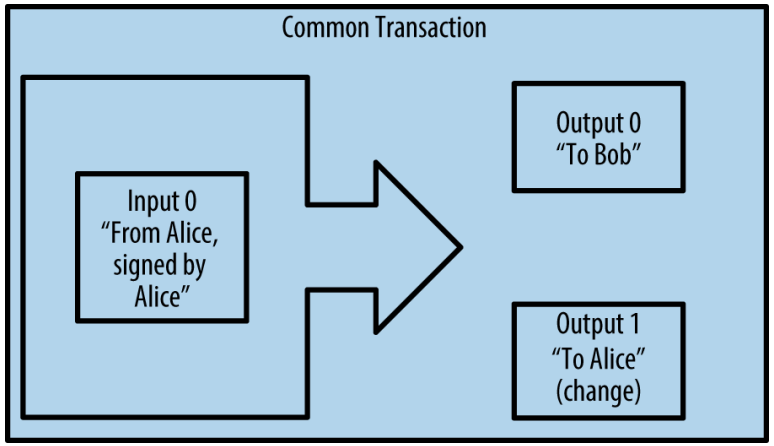
Works offline as well!

Common Transactions

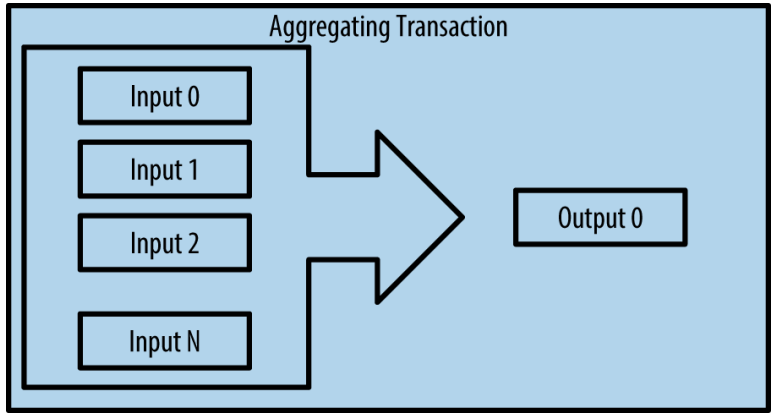
<u>Each input must be used up completely in a transaction</u>

Input that is not spent goes back to sender

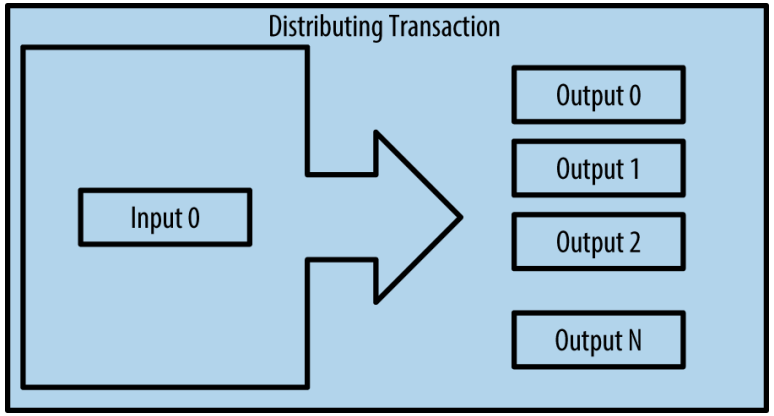
Each transaction can be used exactly once, to avoid double spending



Paying coffee and receiving change



Collecting income



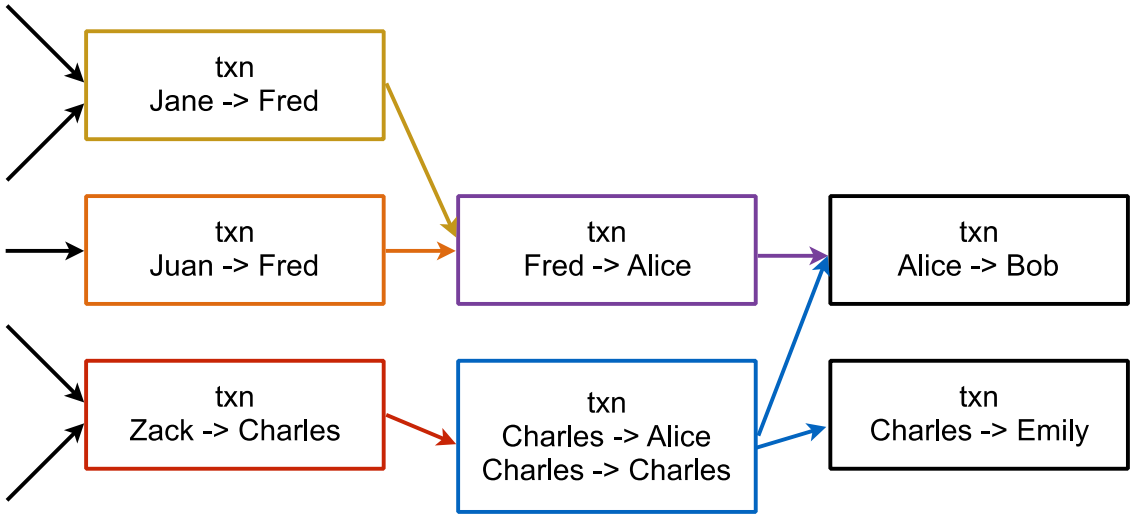
Distributing income

Validity check

Validity of each transaction dependent on the validity of previous transactions

After first installation of wallet software

- Download of each transaction ever made
- Software checks validity back to the very first transaction

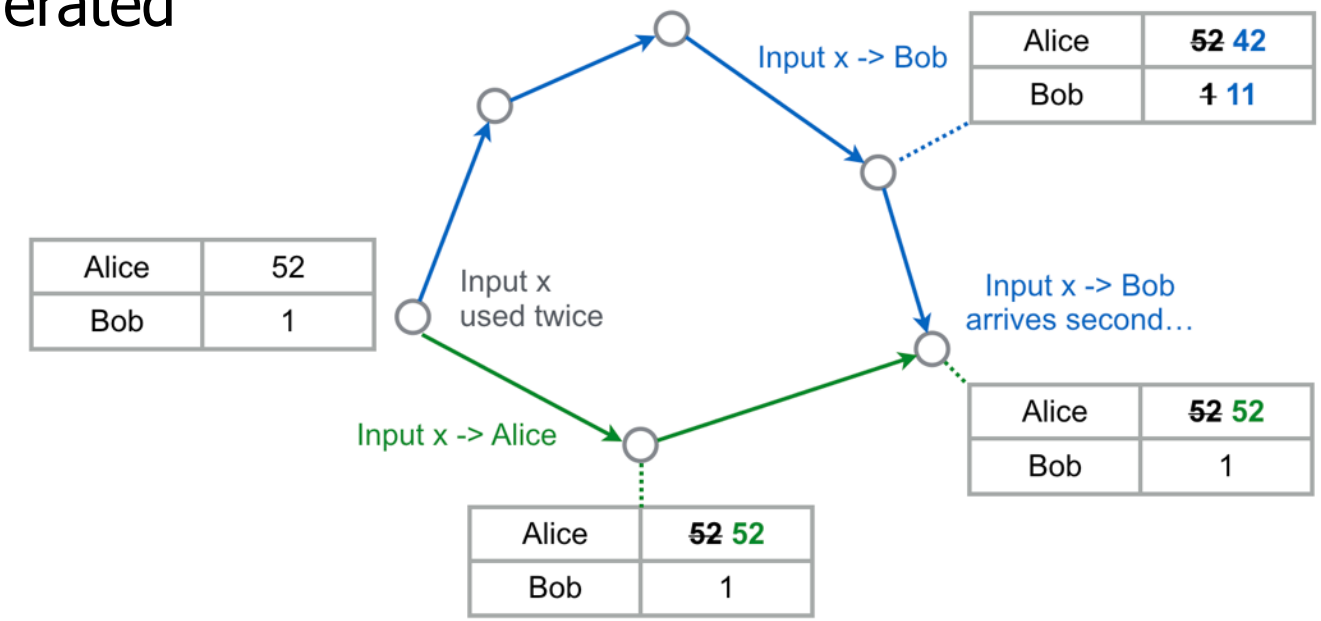


Jane -> Fred	…
Juan -> Fred	…
Zack -> Charles	…
Fred -> Alice	…
Charles -> Alice	…
Charles -> Charles	…
Alice -> Bob	…
Charles -> Emily	…

Transaction order

Node-by-node propagation of transaction messages

Order in which they are received does not have to represent the order in which they were generated



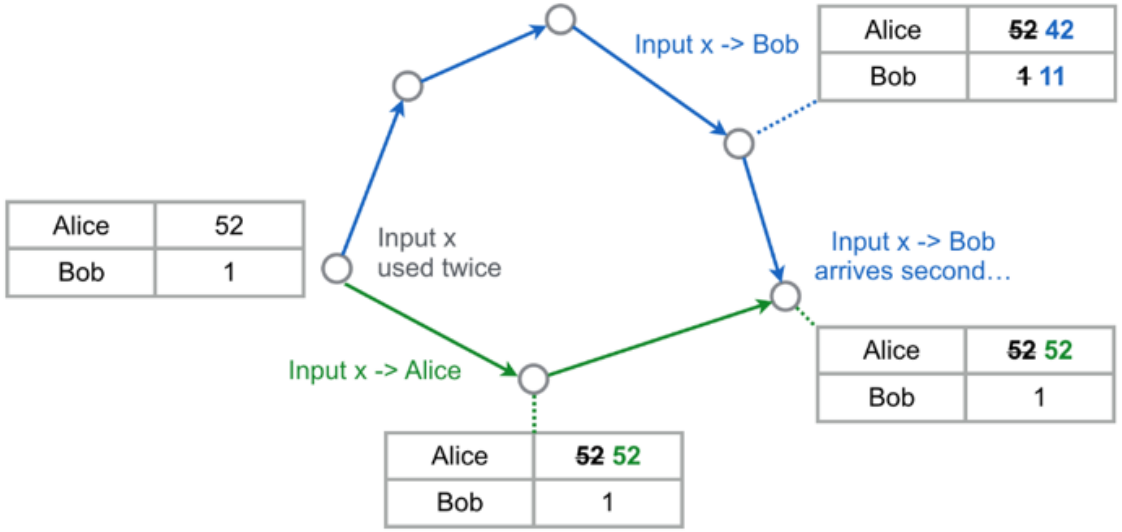
Transaction order: double spending attack

Malicious user Alice could spend input twice

- Once to Bob
- A second time to itself

Results

- Reused input (i.e. input arriving second) considered invalid
- Disagreement in the network



Bob might have shipped the product before noticing second transaction

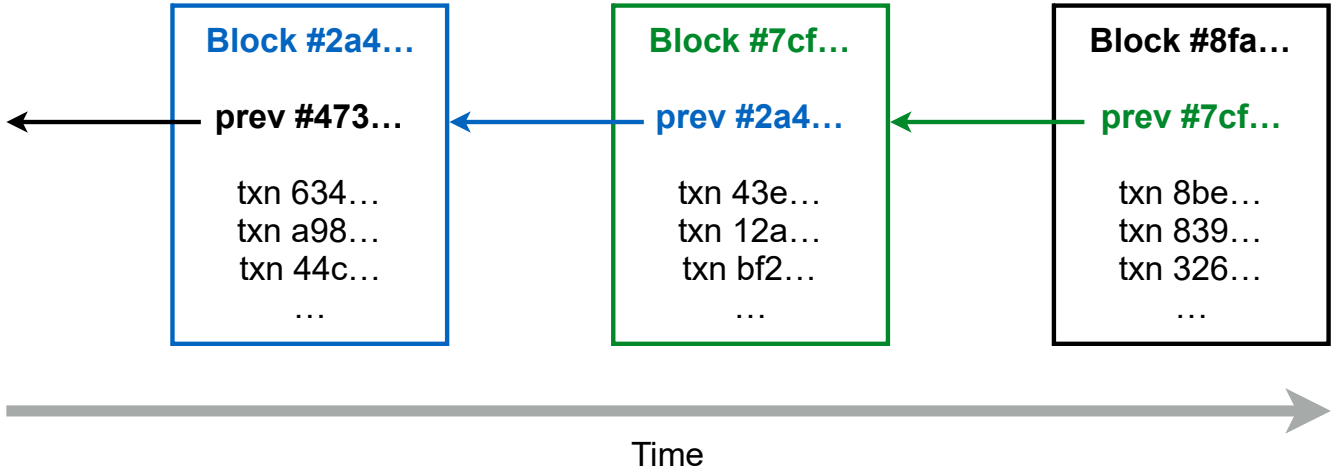
Need for entire network to agree about the order of transactions

Solution: transaction in longest chain wins

Grouping transaction in blocks

Connecting blocks to previous blocks: Placing them one after another in time

All blocks can be traced backwards to the very first transaction or block

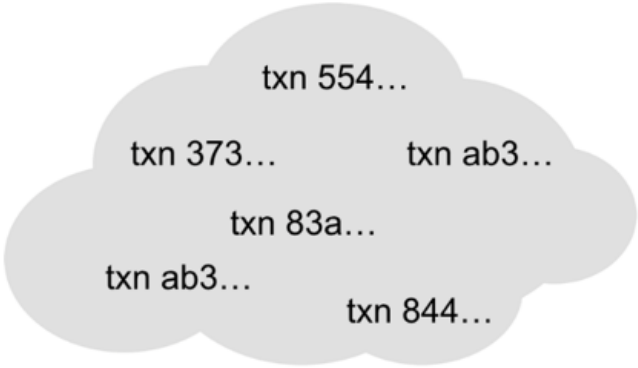


Solution: ordering transactions in blocks II

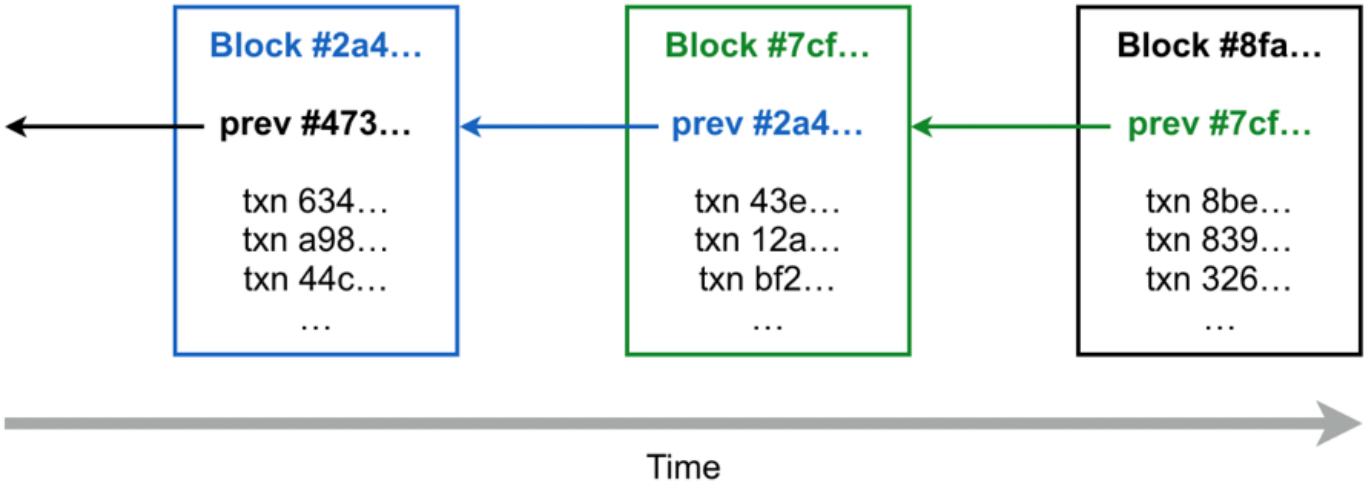
Transactions in same block: considered to happen at same time

Transactions not in a block: unconfirmed/unordered transactions

How to agree on how to group the transactions in a new block?



Unconfirmed/ unordered transactions



Grouping transactions together in blocks

The number of blocks depends on different factors:

- Generally dependent on miner
- Transaction fees (more transactions with fees generate greater payoff)
- Propagation times (smaller block propagate quicker)

Number is generally growing

Ordering of transactions through "block puzzle"

Each block must contain the answer to a mathematical problem that is hard to solve

Proof-of-Work: Block including a random guess (nonce) is run through cryptographic hash (SHA 256) until the output is below a certain value in order to solve a block

- Solution (random guess) can be easily verified
- Used as block reference to identify previous block

Block #8fa…

prev #7cf…

txn 8be…
txn 839…
txn 326…
…

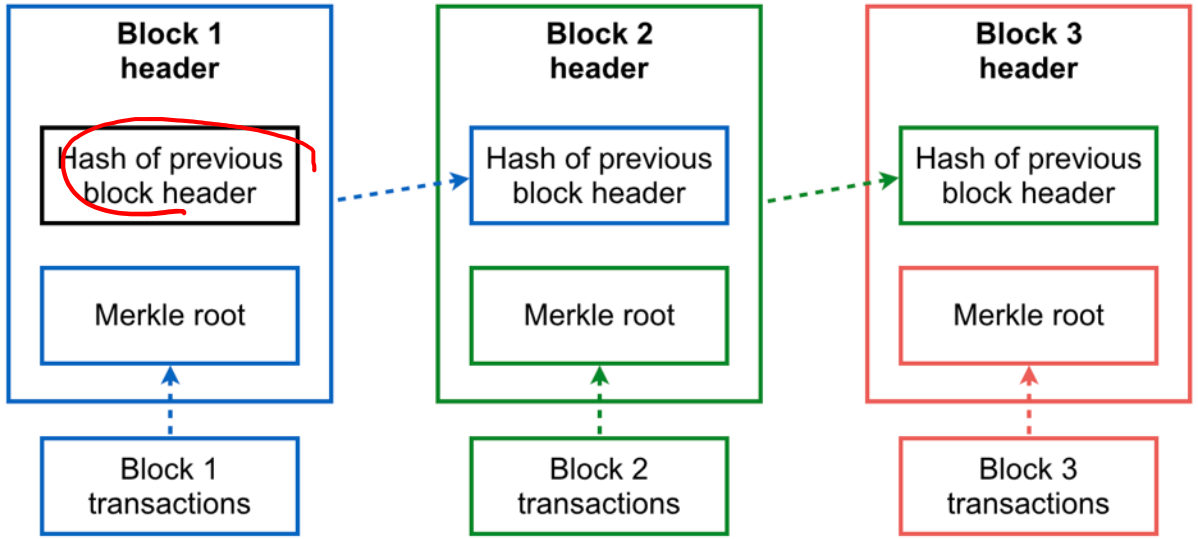
random guess
1. 30282938
2. 30282939
3. 30282940
…

Condition for solution:
f(block)<target

The block structure

Block of one or more transactions collected into the transaction data part of the block

Each block stores hash of previous blocks chaining them together



The block header structure

Size	Field	Description
4 bytes	Version	A version number to track software/protocol upgrades
32 bytes	Previous Block Hash	A reference to the hash of the previous (parent) block in the chain
32 bytes	Merkle Root	A hash of the root of the merkle tree of this block's transactions
4 bytes	Timestamp	The approximate creation time of this block (seconds from Unix Epoch)
4 bytes	Difficulty Target	The proof-of-work algorithm difficulty target for this block
4 bytes	Nonce	A counter used for the proof-of-work algorithm

Detailed breakdown proof-of-work

1. Hashing
   - Miners take the data from a new block (including transaction data, a timestamp, a nonce, and the previous block's hash) and run it through a cryptographic hash function SHA-256. The hash function produces a unique, fixed-size output (the hash) for any given input.
2. Puzzle
   - The hash output is checked against a target value, which is determined by the network's difficulty level. This target value is very low, requiring a lot of trial and error to find a hash that meets the condition.
3. Finding the Solution
   - Miners adjust a "nonce" (a random number) in the block's data and re-hash it repeatedly until they find a hash value that is below the target. The first miner to find a valid hash is rewarded with cryptocurrency.
4. Verification
   - Solution and block broadcasted to the rest of the network. Other nodes verify the solution by running the hash function and checking if the output meets the difficulty target. If it does, they accept add the block to their copy of the blockchain.

Example: trying until hash with a single leading 0 found

I am Satoshi Nakamoto0 → a80a81401765c8eddee25df36728d732...
I am Satoshi Nakamoto1 → f7bc9a6304a4647bb41241a677b5345f...
I am Satoshi Nakamoto2 → ea758a8134b115298a1583ffb80ae629...
I am Satoshi Nakamoto3 → bfa9779618ff072c903d773de30c99bd...
I am Satoshi Nakamoto4 → bce8564de9a83c18c31944a66bde992f...
I am Satoshi Nakamoto5 → eb362c3cf3479be0a97a20163589038e...
I am Satoshi Nakamoto6 → 4a2fd48e3be420d0d28e202360cfbaba...
I am Satoshi Nakamoto7 → 790b5a1349a5f2b909bf74d0d166b17a...
I am Satoshi Nakamoto8 → 702c45e5b15aa54b625d68dd947f1597...
I am Satoshi Nakamoto9 → 7007cf7dd40f5e933cd89fff5b791ff0...
I am Satoshi Nakamoto10 → c2f38c81992f4614206a21537bd634a...
I am Satoshi Nakamoto11 → 7045da6ed8a914690f087690e1e8d66...
I am Satoshi Nakamoto12 → 60f01db30c1a0d4cbce2b4b22e88b9b...
I am Satoshi Nakamoto13 → 0ebc56d59a34f5082aaef3d66b37a66...

Level (#leading zeros) is increasing every 2016 blocks to meet 10 min average

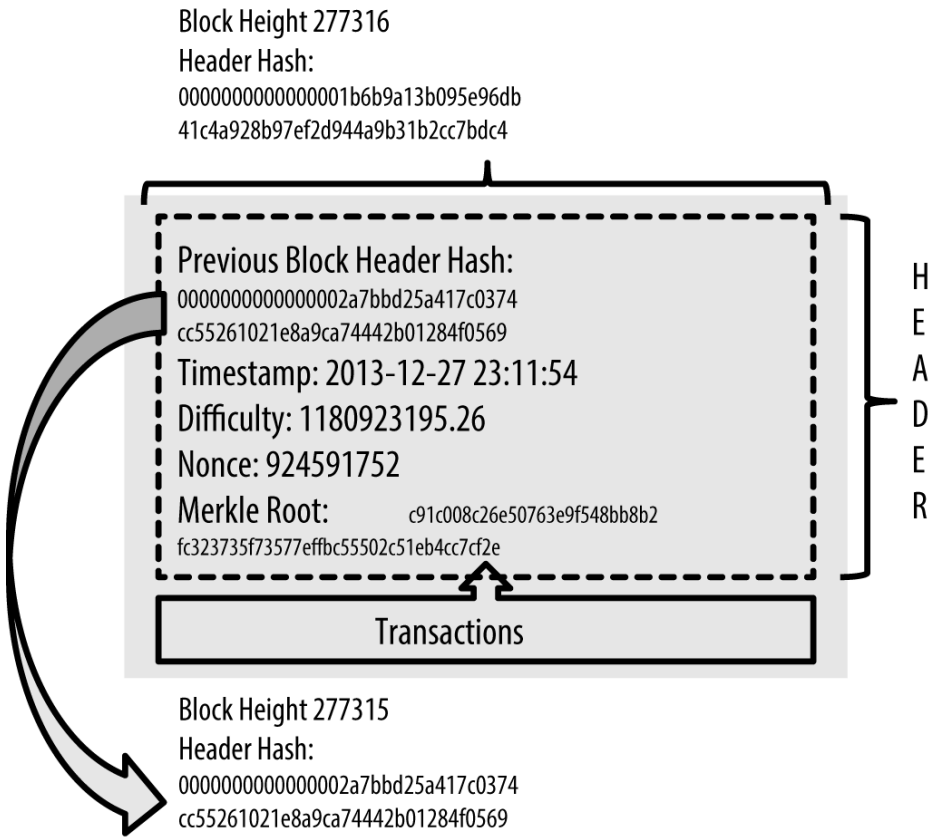
Merkle root

Summary of all the transactions in the block

Root of Merkle tree: Binary tree, containing cryptographic hashes

Efficient summary and verification

Algorithm used: SHA256 applied twice (double-SHA256)



Challenge: blocks solved at the same time

Solving a block

- For one computer: several years
- For the network: 10 minutes
- Compromise between stability and confirmation time

Randomness spreads out the time for finding a solution

First computer to solve a block broadcasts it, group of transactions is accepted as next in blockchain

But how to cope with near simultaneous solutions?

Block #8fa…

prev #7cf…

txn 8be…
txn 839…
txn 326…
…

random guess
30282940
…

f(block)<target

Block #8fa…

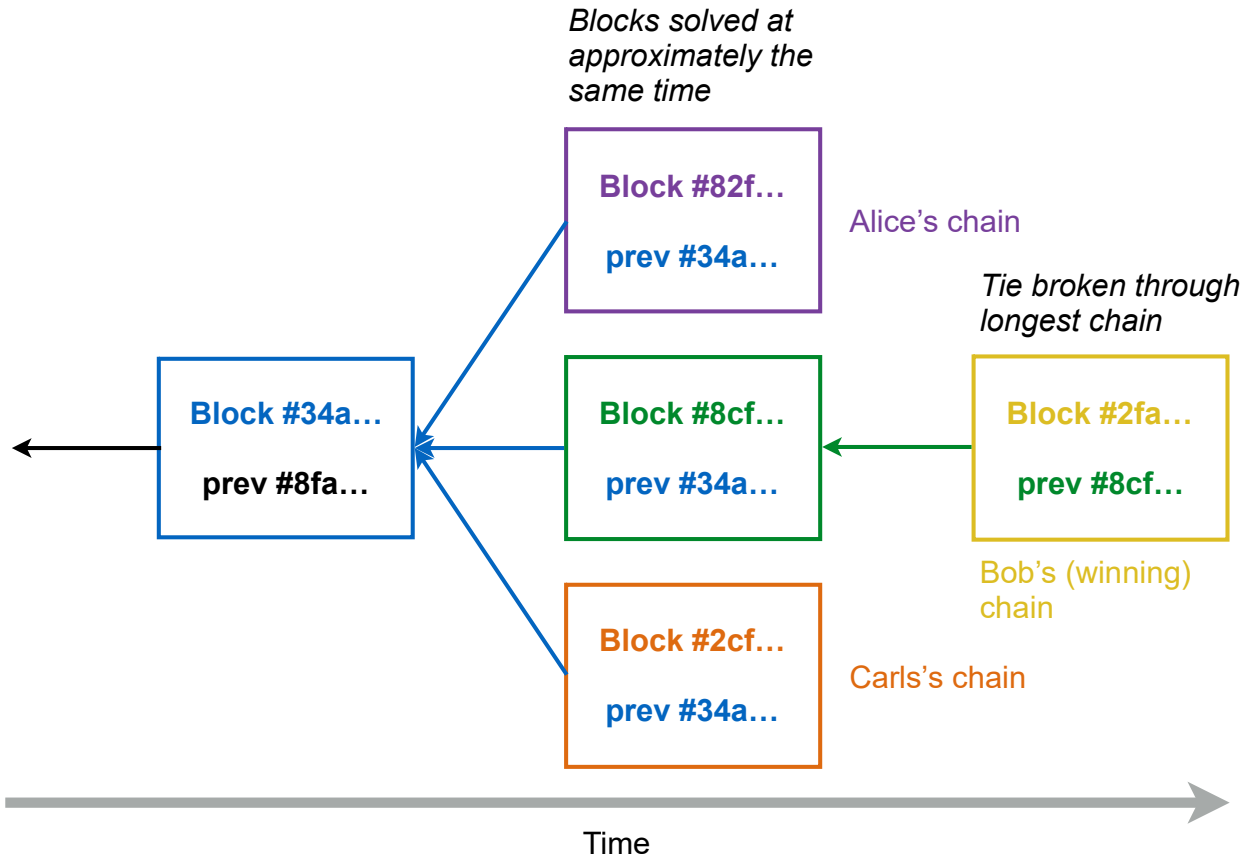
prev #7cf…

txn 8be…
txn 839…
txn 326…
…

random guess
89217341
…

f(block)<target

Longest chain wins and Blockchain stabilizes quickly

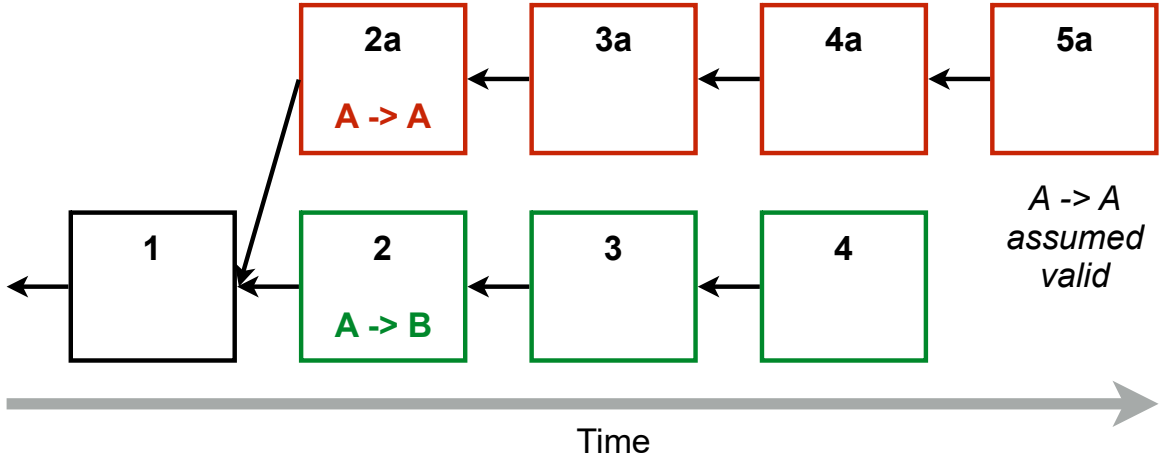


Double spending attach revisited

Attack still possible, as long as the longest branch can be presented

Example

- Bob (B) shipped product to Alice (A) before 4a and 5a presented
- Once block 5a is presented, transaction A → B becomes invalid



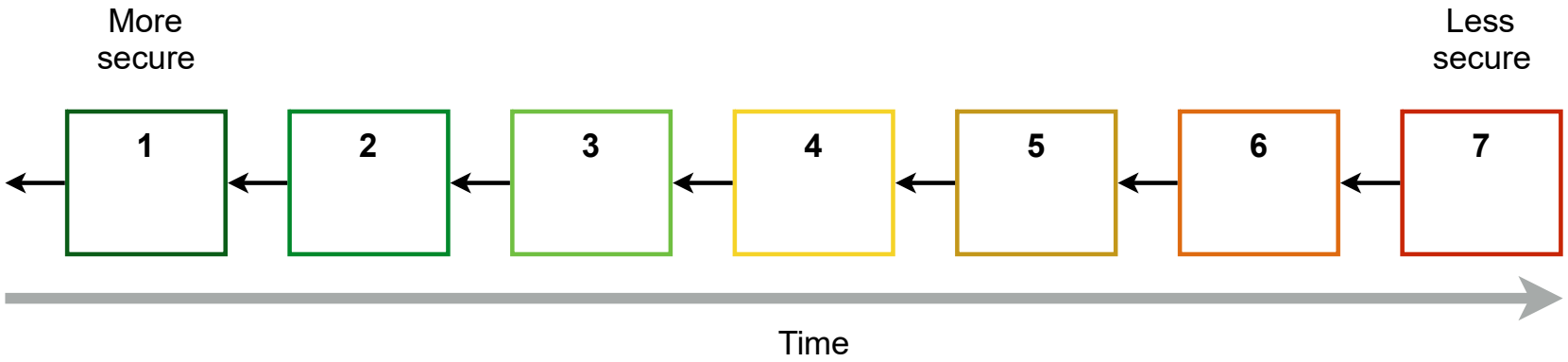
Double spending attach revisited II

Unlikely for Alice (generating red branch on the previous slide) to outrace the network

- 50% of computing power needed to have a 50% chance of solving a block before someone else does

Race: attacker vs. the entire rest of the network

Transactions in the back of the chain are more secure



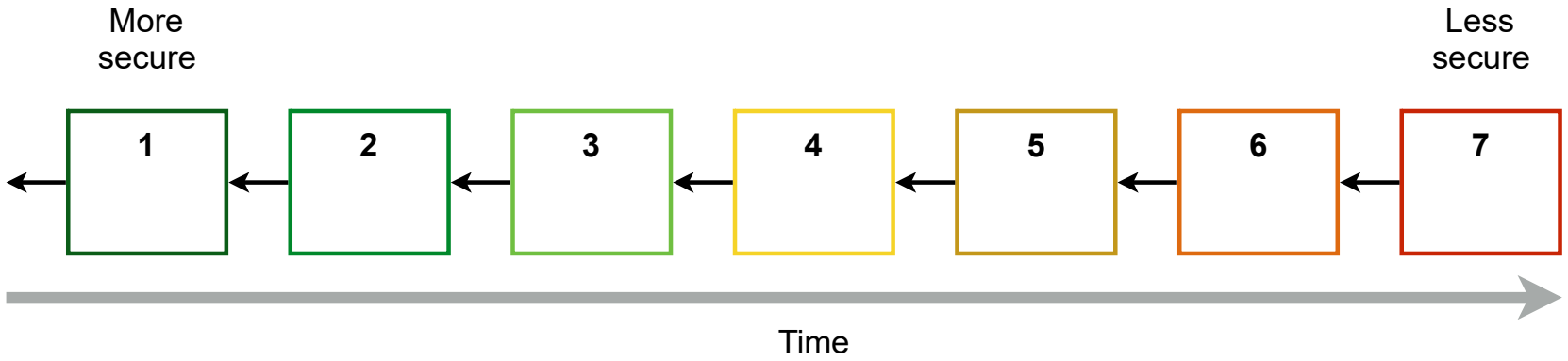
Double spending attach revisited

Keep in mind: block in-between cannot be exchanged, since then all subsequent hashes would be invalidated

Vulnerability towards the front (later) end of the chain

Solution: wait for a certain amount of blocks before considering the received money final

- Current recommendation: **six blocks (corresponding to 1 h) back** → Still much faster than credit cards from the merchant's perspective



Origin of bitcoins

Reward given to whomever solves a block: Solving blocks: “mining”

Real purpose: Verify and safeguard (order of) transactions

Every four years: block reward cut in half

Further incentive to mine blocks: Miners receive transaction fees that are optionally included with transactions

- Transaction processing likely in order (decreasing) of the fees attached to transactions
- Transactions without fees possibly ignored (in the future)

Every two weeks difficulty of math problem is recalibrated to target 10 min

Selected aspects in depth

Linking transactions together

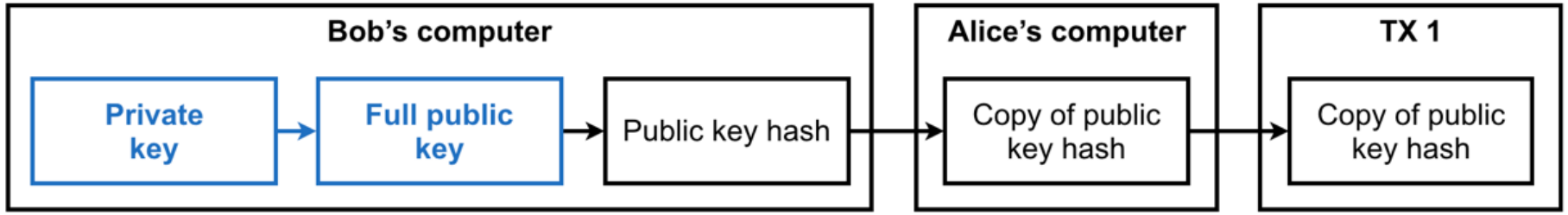
Two parts

- Alice obtains Bob's public key hash and sends him bitcoins
- Bob spends the bitcoins

Both use most common form of the Pay-To-Public-Key-Hash (P2PKH) transaction type

Step 1: Bob generates a private/public key pair

In order for Alice to be able to create first transaction



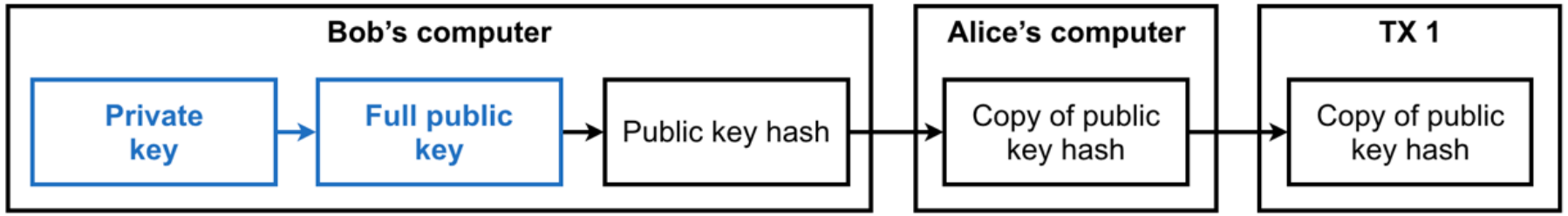
Step A.1: private/public key generation

Algorithm used: Elliptic Curve Digital Signature Algorithm (ECDSA)

Variant of digital signature algorithm (DSA), with secp256k1 curve

Private key: 256 bits of random data

- Deterministically transformed into public key: No need to store it, since it can be generated at any time as long as private key is available



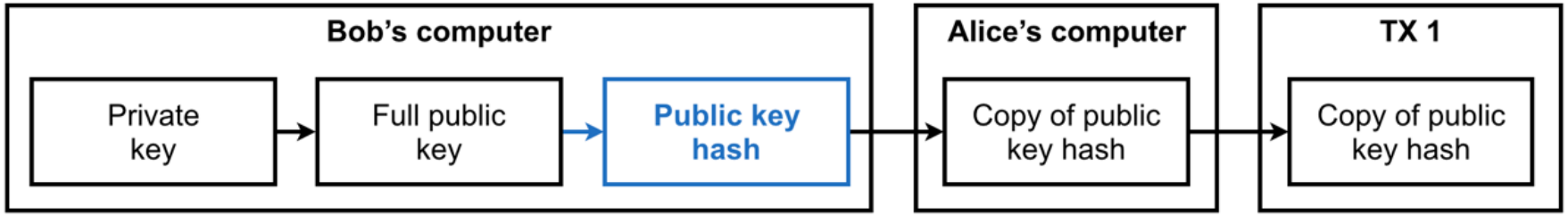
Step A.2: public key hash generation

Short “pubkey hash”

Can be reliably repeated later, also does not need to be stored

Shortens and obfuscates public key

- Easier manual transcription
- Security against unanticipated problems that might allow reconstruction of private keys from public key data at later point in time

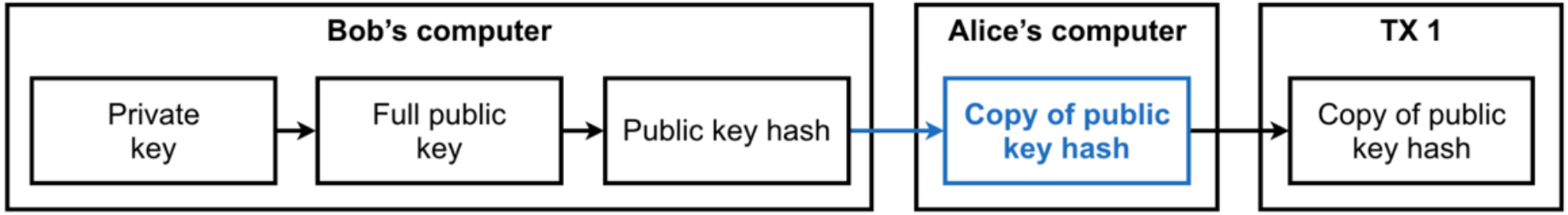


Step A.3: transmission of pubkey hash to Alice

Encoded as Bitcoin address

- Using base58 encoding: A–Z, a-z, 0-5, but omitting e.g. 0, O, I and l: 123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz
- Containing further Address version number and Checksum

Can be transmitted through any medium, even one-way medium



Step A.4: Alice creates a transaction

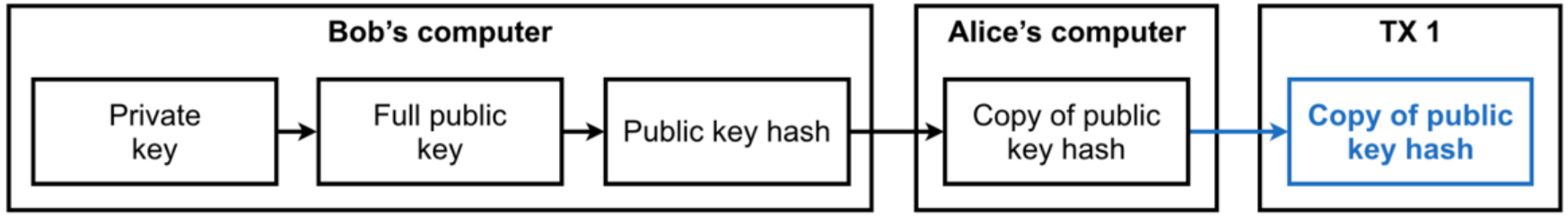
Decodes received address into standard hash (pubkey hash)

Create standard P2PKH transaction output

- Including instructions, so that everyone that proves control of the private key matching Bob's hashed public key can spend the output, also called pubkey script transactions

Alice broadcasts the transaction and it is added into blockchain

- Categorized by network as unspent transaction output (UTXO)
- Bob's wallet software displays it as spendable balance

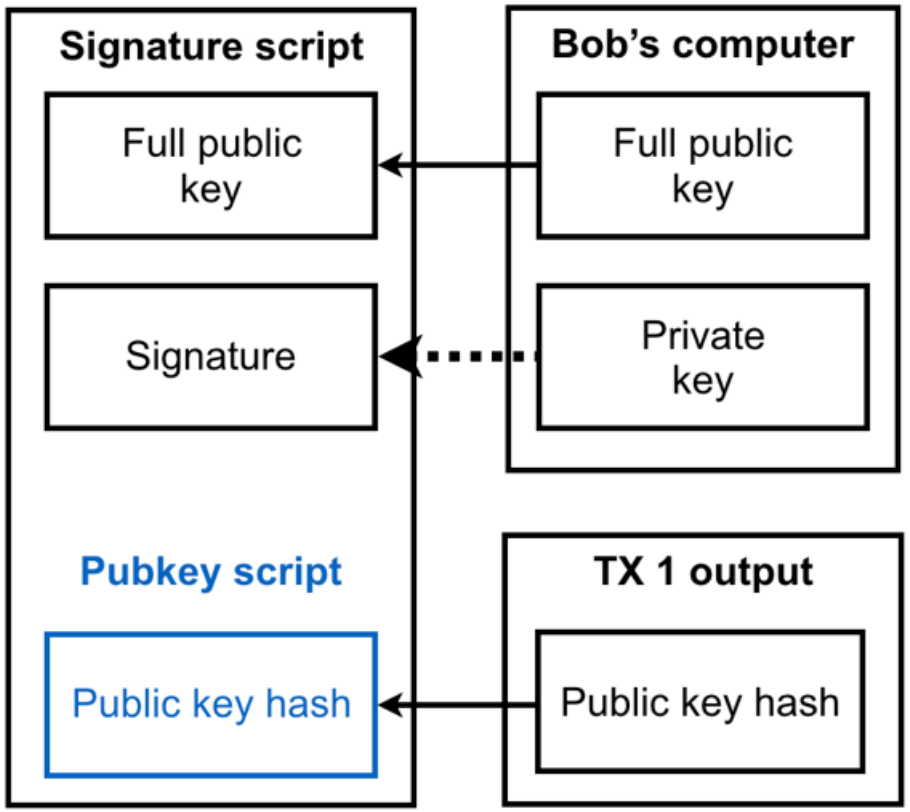


Step B.1: Bob reference previous transaction, creates pubkey script

Spendable balance that arrived from Alice is also called unspent transaction output (UTXO)

In order to spend Alice's output

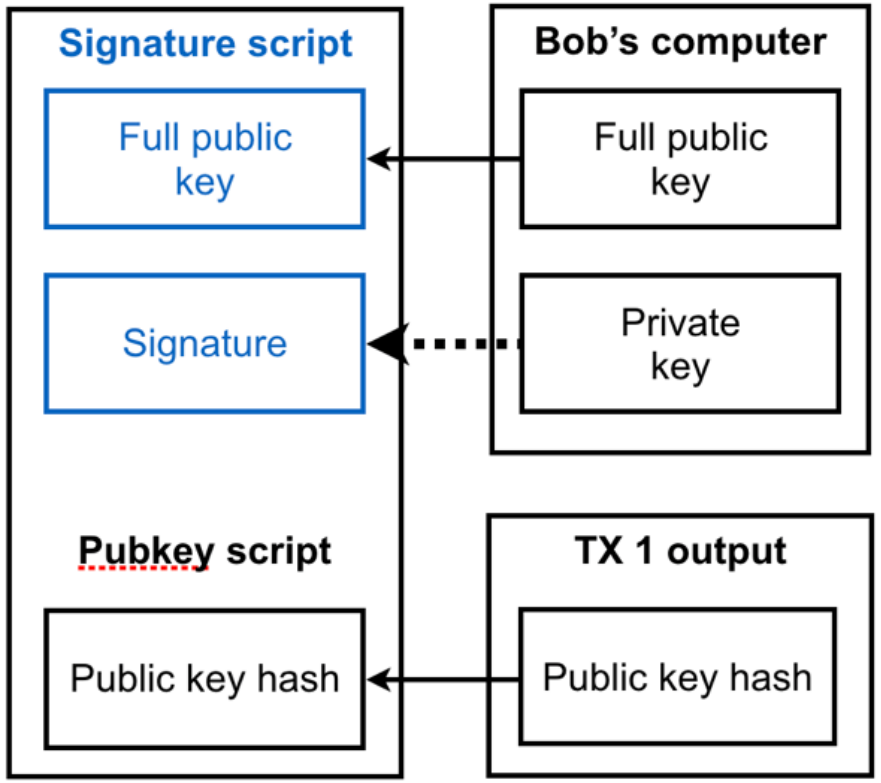
- Bob references the transaction Alice created by its hash (called transaction identifier, txid) and the specific output by index number (not depicted)
- … creates a pubkey script
  - Defining what conditions have to apply in order for the amount to be spent



Step B.2: Bob creates signature script

Bob creates a signature script (also called scriptSigs): collection of data satisfying the conditions Alice placed into her previous output's pubkey script

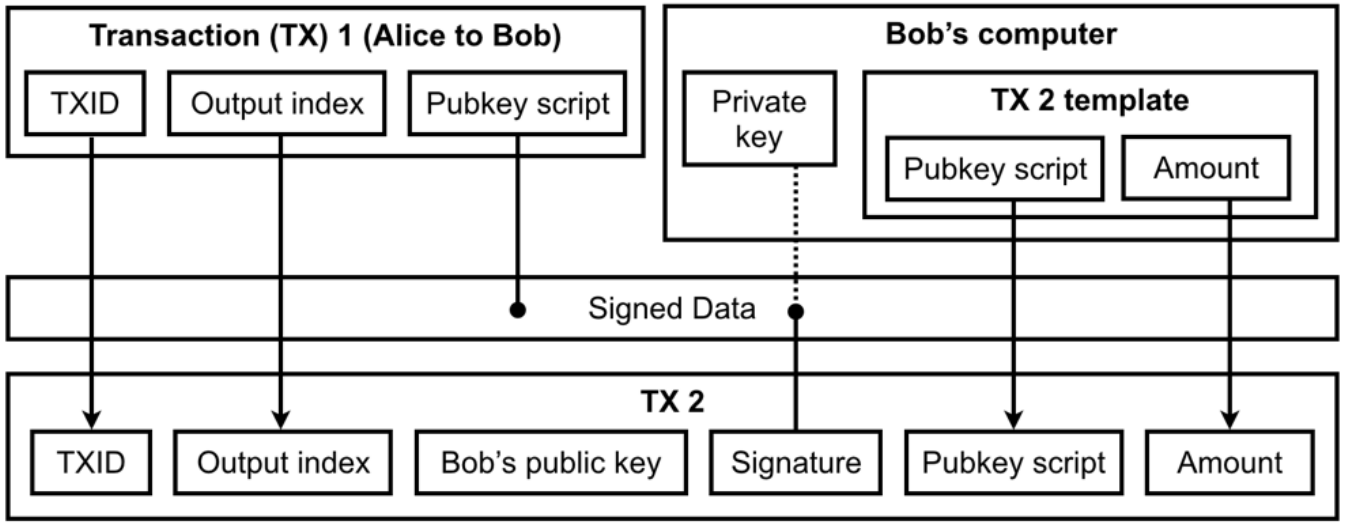
- Bob's unhashed public key (can be checked for conformance with pubkey hash)
- Proves that Bob controls his private key
- Makes non-signature-script parts of his transaction tamper-proof (can be safely transmitted)



Step B.3: Putting it together

Data signed by Bob includes:

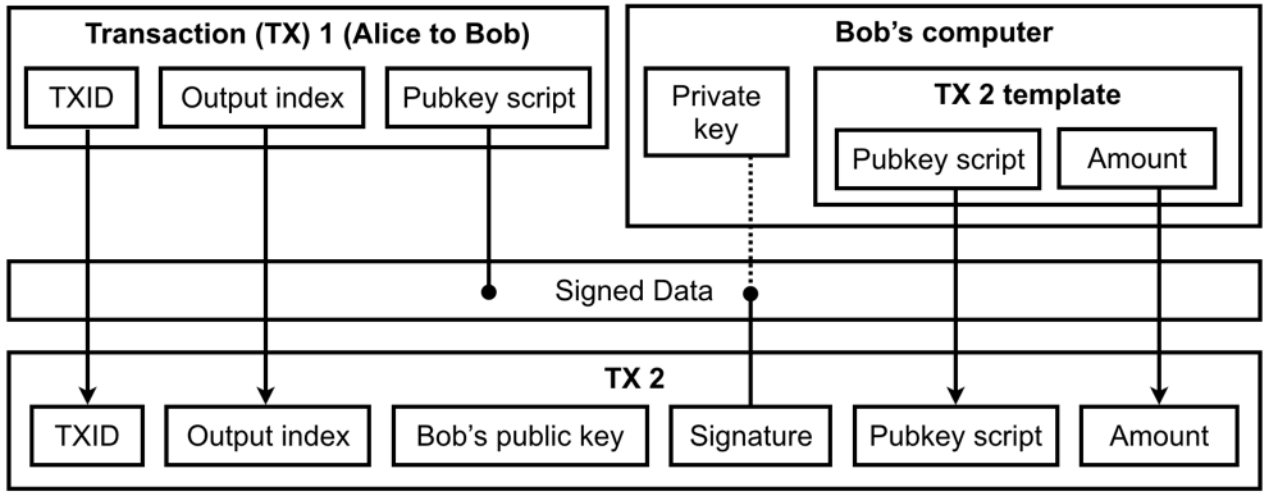
- TXID, output index of previous transaction, previous outputs pubkey script (all from TX1)
- Bob's pubkey script let's next recipient spend the amount, the amount itself (all from TX2 template)



Step B.3: Putting it together

Entire transaction signed except for any signature scripts that hold full public keys and signatures

After assembling TX2, Bob broadcasts it to the network to be included in the blockchain



Questions ?